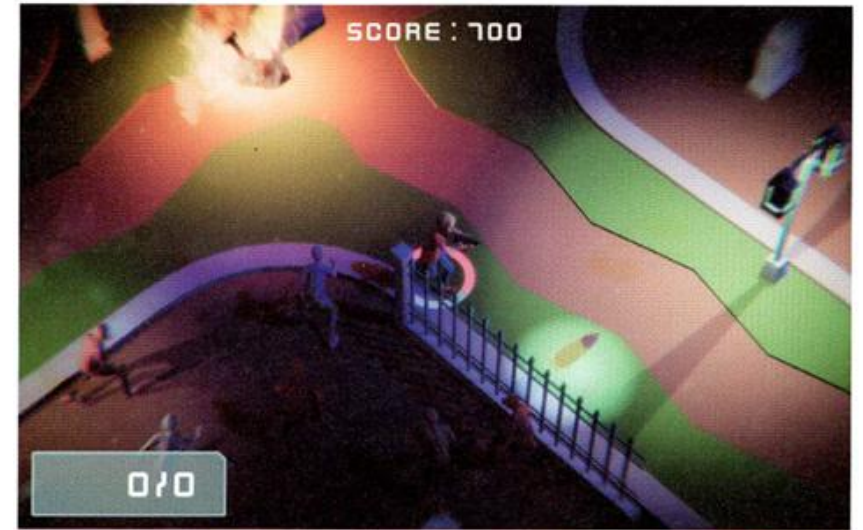


좀비 서바이버

- 미션 : 좀비들이 끊임없이 접근, 기관총으로 쏘면서 점수를 모으고 가능한 한 오래 살아남기
- 기능
 - 좀비는 일정주기로 생성, 시간이 지날수록 한번에 생성되는 좀비 수 증가 (프리팸)
 - 좀비는 플레이어의 위치를 주기적으로 파악하고 언제나 최적의 경로로 추적 (AI)
 - 좀비의 이동속도와 공격력, 체력은 랜덤으로 지정 (랜덤)
 - 강하고 빠른 좀비일 수록 피부 색깔이 붉어짐 (클래스 상속)
 - 플레이어의 체력은 캐릭터를 따라다니는 원형바로 확인 (UI)
 - 플레이어의 기관총 탄알은 무한하지 않고, 체력도 자동 회복 X 탄알 아이템과 체력 아이템 찾아야 함
 - 아이템은 주기적으로 플레이어 근처의 랜덤한 위치에 생성 일정한 시간 뒤에 사라짐
 - 후처리(Post-Processing) 효과를 사용해 게임 화면 보정
 - 유니티 공식 패키지 사용 (패키지 매니저, 시네머신, 포스트-프로세싱 스택)



- 조작법

- 캐릭터 이동 : 화살표키 or WASD
- 발사 : 마우스 왼쪽 버튼
- 재장전 : R

▶ Chapter 14: 좀비 서바이버 레벨 아트와 플레이어 준비

레트로의 유니티 게임프로그래밍 에센스 (개정판)



이제만 지음

Contents

- Chapter 14 좀비 서바이버 레벨 아트와 플레이어 준비
 - 14.1 프로젝트 구성
 - 14.2 레벨 아트와 라이팅 설정
 - 14.3 플레이어 캐릭터와 애니메이션 구성
 - 14.4 캐릭터 이동 구현
 - 14.5 시네머신 추적 카메라 구성하기
 - 14.6 마치며



CHAPTER 14 좀비 서바이버

레벨 아트와 플레이어 준비

- 패키지 매니저
- 라이트 설정으로 씬의 전반적인 색 분위기를 조절하는 방법
- 라이트맵과 글로벌 일루미네이션
- 여러 애니메이션 클립을 섞어 사용하는 방법
- 애니메이션을 특정 신체 부위에만 적용하는 방법
- 플레이어의 입력과 플레이어 캐릭터의 움직임 구현
- 시네머신으로 자동 추적 카메라 만들기

14.1 프로젝트 구성(1)

[과정 01] 좀비 서바이버 프로젝트 열기

① 예제 데이터의 14 폴더 내부의 Zombie 프로젝트 폴더를 유니티로 열기

Assets 폴더

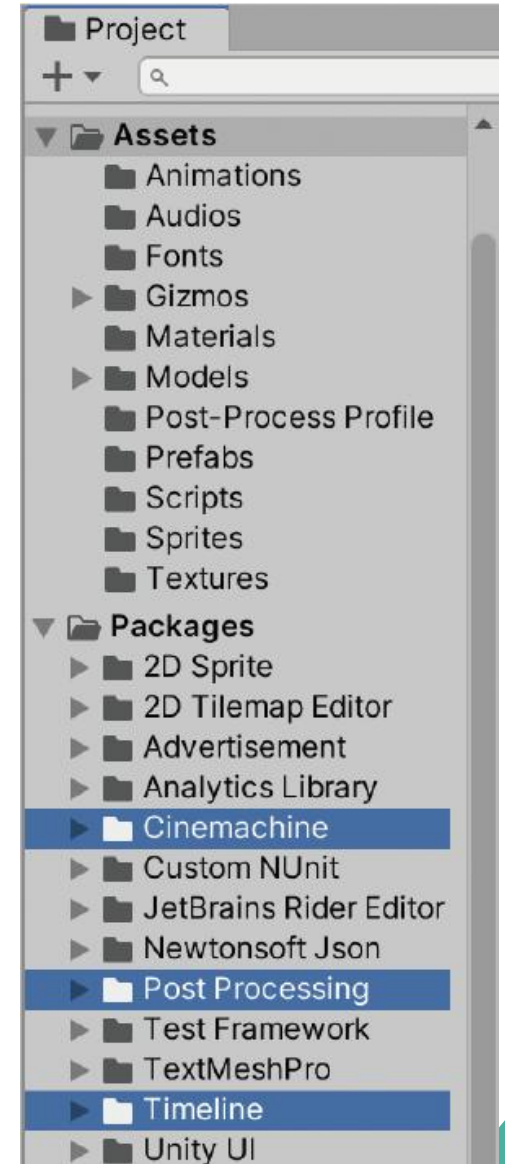
- Animations : 캐릭터 애니메이션
- Audios : 효과음과 음악 오디오 클립
- Fonts : 폰트
- Gizmos : 시네머신이 사용하는 기즈모 아이콘(시네머신 패키지 추가 시 자동 생성됨)
- Materials : 3D 모델에 사용할 머티리얼
- Models : 3D 모델
- Post-Process Profile : 후처리에 사용할 프로파일(프리셋)
- Prefabs : 프리팹
- Scripts : 스크립트
- Sprites : 2D 텍스처(스프라이트)
- Textures : 3D 모델의 텍스처

Packages 폴더

좀비 서바이버 프로젝트에 사용하기 위해 추가한 패키지

- Cinemachine : 스마트 추적 카메라와 복잡한 카메라 연출 쉽게 구현
- Post-process : 후처리 구현
- Timeline : 영상 편집 프로그램과 닮은 시퀀스 편집 툴

실시간 시네마틱 컷 신 등 제작, Cinemachine 패키지에서 필요, 직접 사용 X

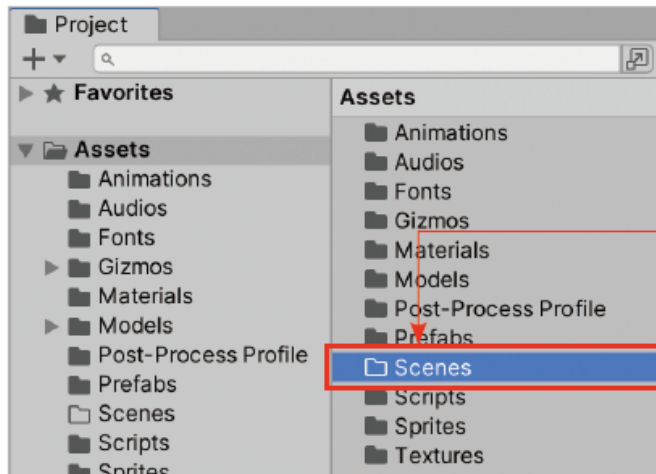


14.1 프로젝트 구성(2)

- 새로운 씬을 만들고 개발을 시작

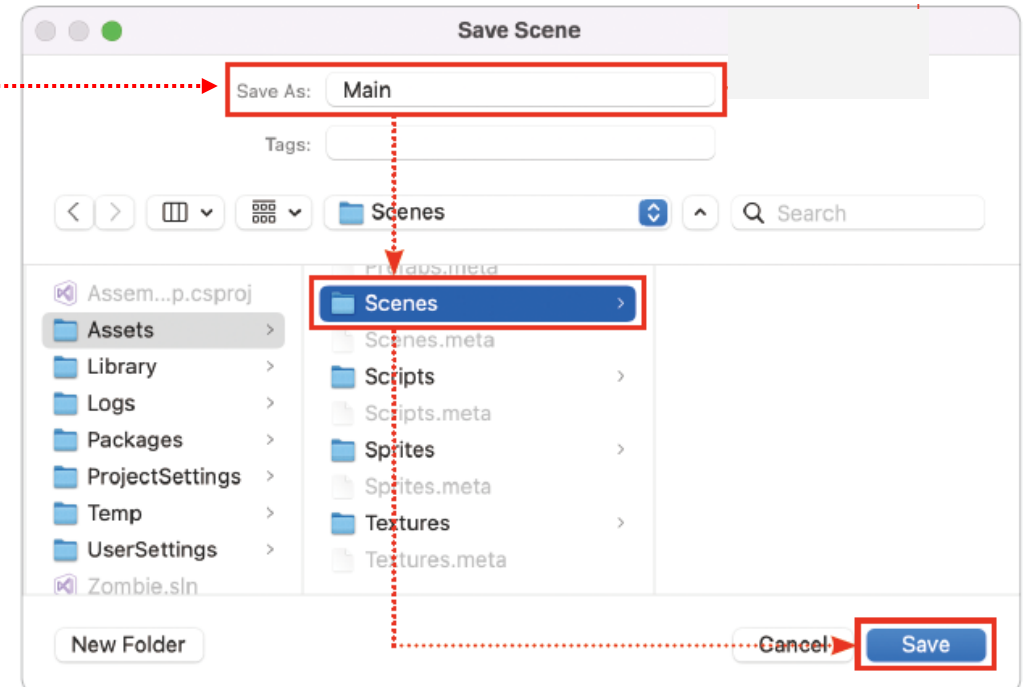
[과정 02] 새로운 씬 만들기

- ① 프로젝트 창의 Assets 폴더에 Scenes라는 이름으로 새 폴더 생성
- ② [Ctrl+S]로 씬 저장 창 열기 > 새 씬을 Main이라는 이름으로 Scenes 폴더에 저장



① Scenes 폴더 생성

② [Ctrl+S]로 씬 저장 창 열기 > 새 씬을 Main이라는 이름으로 Scenes 폴더에 저장



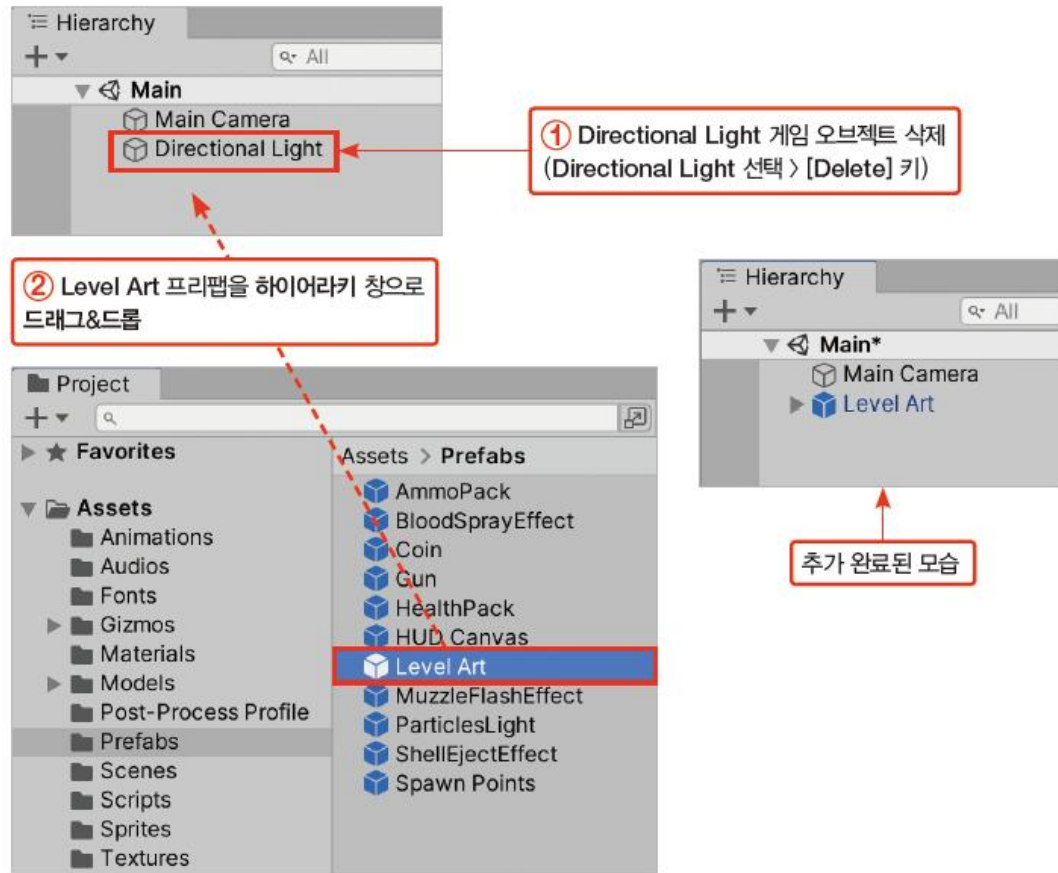
14.2 레벨 아트와 라이팅 설정(1)

14.2.1 레벨 아트 추가하기

- Prefabs 폴더에서 미리 제작된 레벨 아트인 Level Art 프리팹을 찾아 씬에 추가

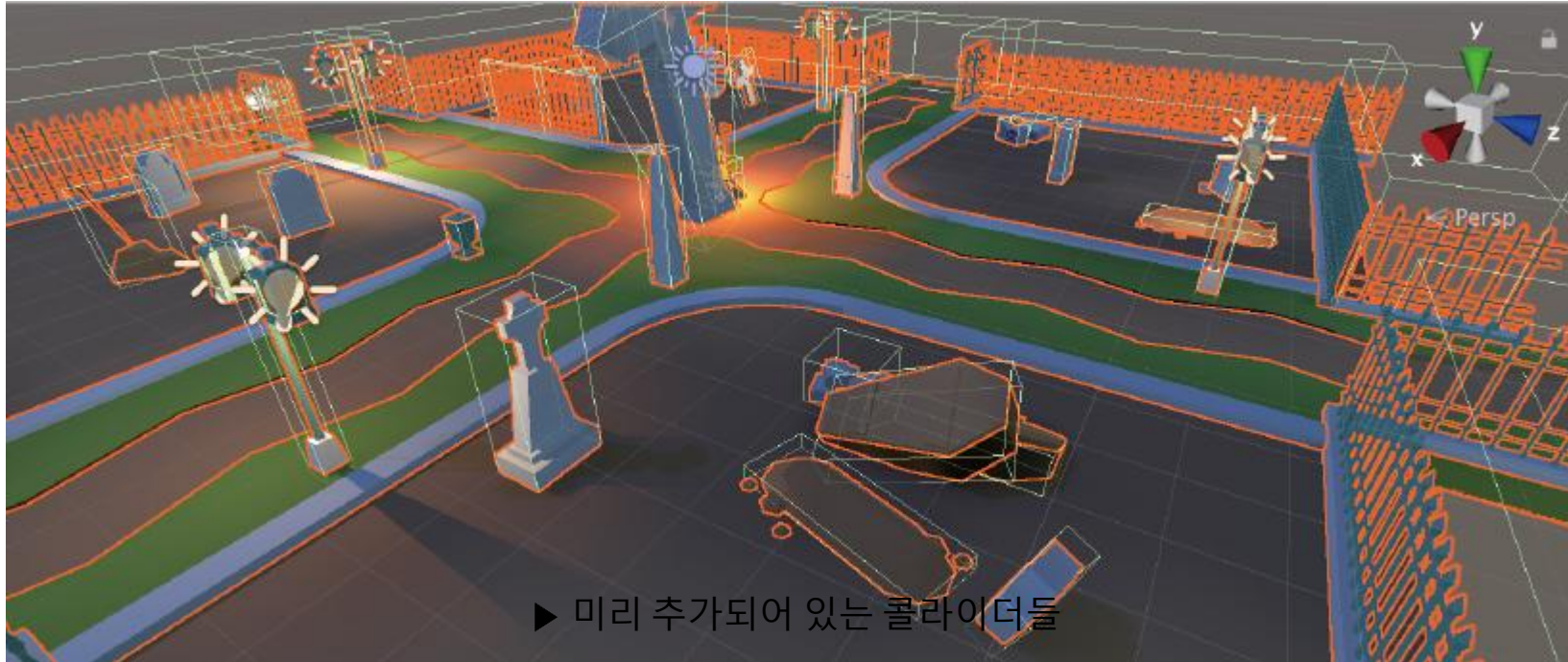
[과정 01] 레벨 아트 구성하기

- ① Directional Light 게임 오브젝트 삭제(Directional Light 선택 > [Delete] 키 누르기)
- ② Prefabs 폴더의 Level Art 프리팹을 하이어라키 창으로 드래그&드롭



14.2 레벨 아트와 라이팅 설정(2)

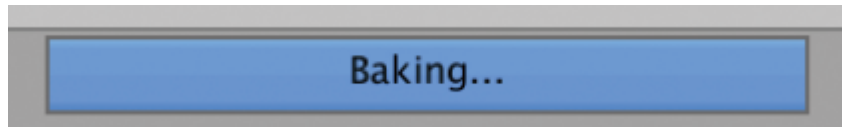
- Level Art 프리팹에는 빛을 내는 라이트 게임 오브젝트가 추가
- 여러 사물과 울타리를 감싸도록 콜라이더 추가
 - 캐릭터가 장애물이나 울타리를 뚫고 지나가는 것을 방지
 - 메시 콜라이더를 사용하면 3D 모델의 모양과 일치하는 콜라이더를 만들 수 있지만 처리량이 증가, 중요한 게임 오브젝트에만 사용하는 게 좋음



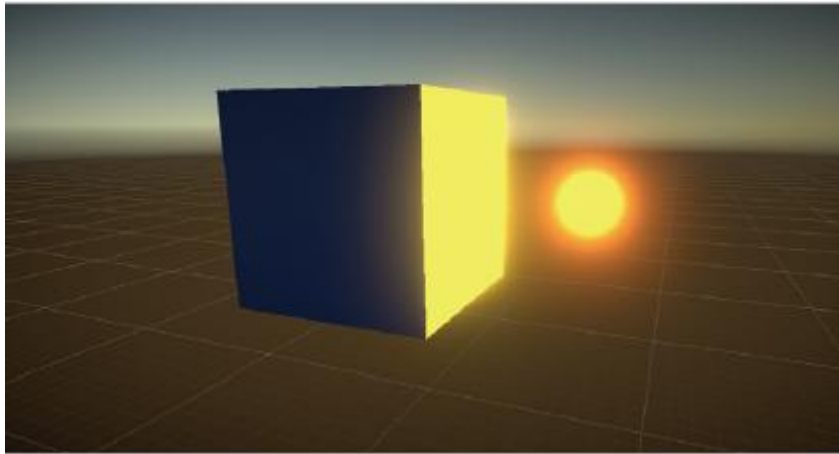
14.2 레벨 아트와 라이팅 설정(3)

14.2.2 라이트맵

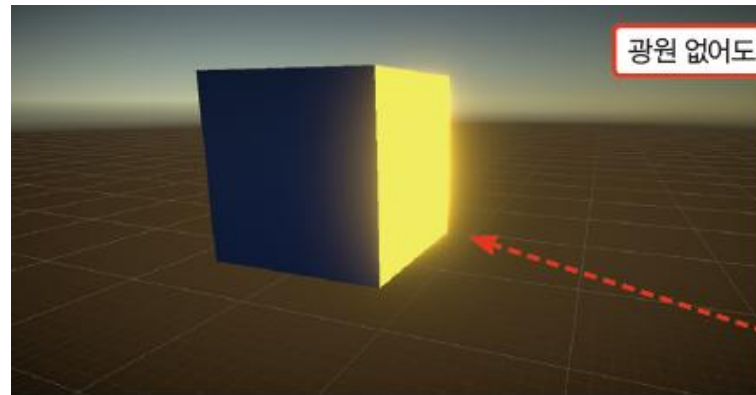
- 유니티는 라이팅 데이터 에셋을 사용하여 라이팅 효과의 실시간 연산량을 줄이며, 씬에 변화가 감지될 때마다 매번 새로운 라이팅 데이터 에셋을 생성
- 라이트맵(Lightmap) - '오브젝트가 빛을 받았을 때 어떻게 보일지' 미리 그려둔 텍스처



▶ 라이트맵 굽기 진행바

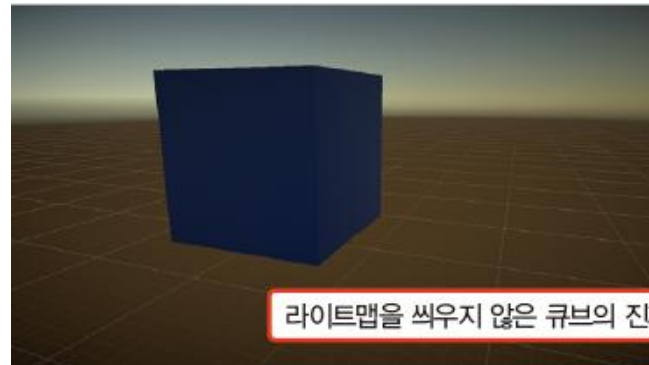


▶ 노란 빛을 받은 큐브

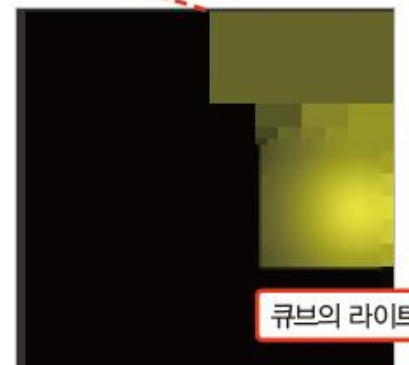


광원 없어도 큐브 벽면이 빛남

큐브에 라이트맵을 덧씌움



라이트맵을 씌우지 않은 큐브의 진짜 모습



큐브의 라이트맵

▶ 라이트맵이 입혀진 큐브

14.2 레벨 아트와 라이팅 설정(4)

14.2.3 라이팅 설정하기

- 유니티 상단 메뉴에서 Window > Rendering > Lighting Settings로 들어가 라이팅 설정 창 열기

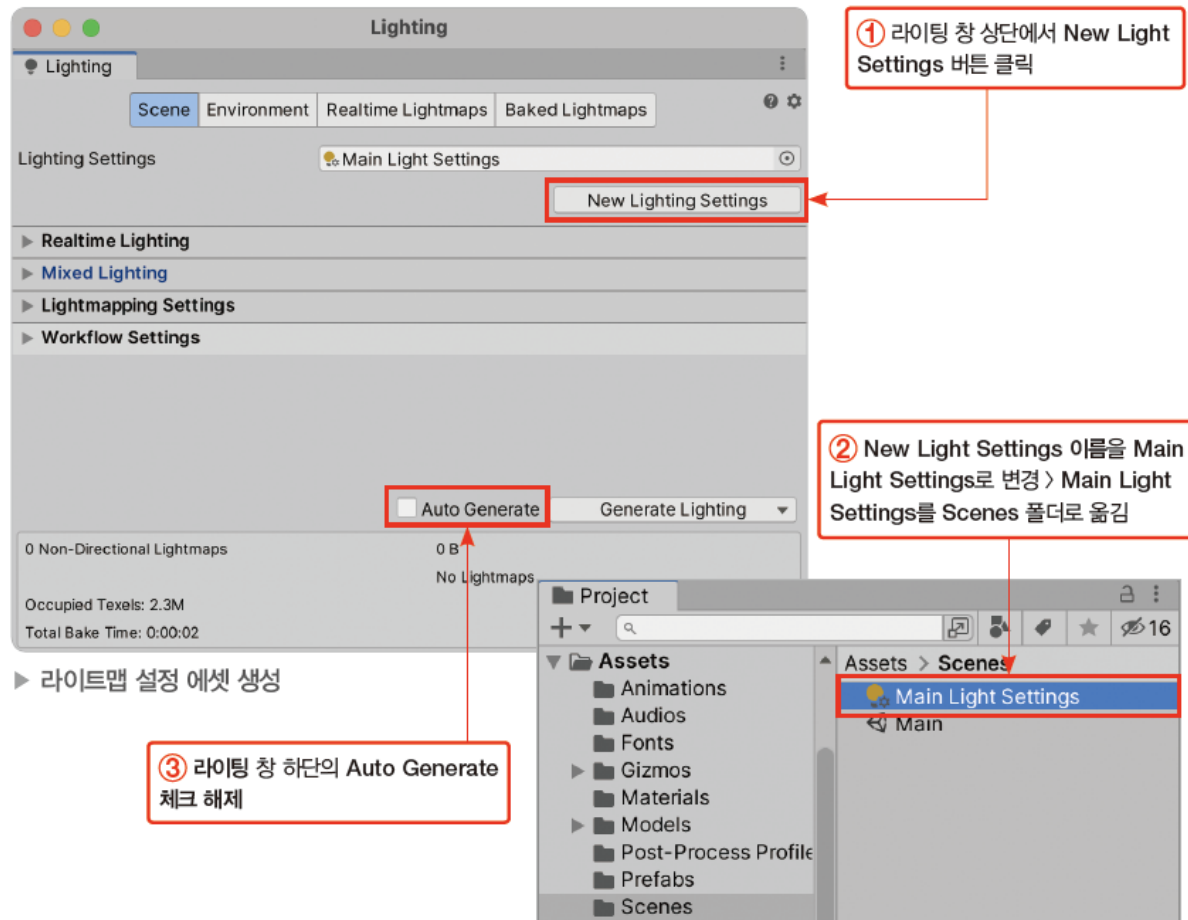


14.2 레벨 아트와 라이팅 설정(5)

- 현재 씬을 위한 새로운 라이트 설정 에셋을 생성

[과정 01] 라이트맵 설정 에셋 생성

- ① 라이팅 창 상단에서 New Light Settings 버튼 클릭 > 라이트 설정 에셋 New Light Settings가 생성됨
- ② New Light Settings 이름을 Main Light Settings로 변경 > Main Light Settings를 Scenes 폴더로 옮김
- ③ 라이팅 창 하단의 Auto Generate 체크 해제



14.2 레벨 아트와 라이팅 설정(6)

- 환경광(Environment Lighting)설정 - 환경광은 씬에 가장 기본으로 깔리는 빛 (그림자나 명암 생성 X)

[과정 02] 환경광 설정하기

- ① 라이팅 창 상단의 Environment 탭 클릭 > Enviroment 패널에서 Source를 Color로 변경
- ② Ambient Color의 컬러 필드 클릭 > 컬러를 (65, 23, 12)로 변경

① Environment Lighting의 Source를 Color로 변경

② Ambient Color의 컬러 필드 클릭 > 컬러를 (65, 23, 12)로 변경

환경광 설정하기

HDR Color

RGB 0-255

R 65

G 23

B 12

Intensity 0

Swatches

Click to add new preset

14.2 레벨 아트와 라이팅 설정(7)

14.2.4 글로벌 일루미네이션

- 글로벌 일루미네이션(Global Illumination, GI)
 - 물체의 표면에 직접 들어오는 빛뿐만 아니라 다른 물체의 표면에서 반사되어 들어온 간접광까지 표현

실시간 글로벌 일루미네이션 (진정한 실시간은 아님)

- 빛의 세기와 방향 등이 달라졌을 때 그 변화를 간접광에 실시간으로 반영
 - 라이트맵을 여러 방향에 대해 생성
 - 여러 가지 경우에 대한 빛의 예상 반사 방향과 광원의 예상 이동 경로 등의 정보를 **미리** 계산해서 저장

베이킹된 글로벌 일루미네이션

- 고정된 빛에 의한 간접광들을 라이트맵으로 구워 게임 오브젝트 위에 미리 입힘
- 반영된 간접광 효과는 게임 도중에 실시간으로 변하지 않음
- 라이팅 모드
 - 베이킹된 간접(Baked Indirect) 모드
 - 섀도우마스크(Shadowmask) 모드 – 기본값
 - 감산(Subtractive) 모드

두 설정 모두 빛이 입혀질 오브젝트는 고정되어 있어야 함

- 베이크된 간접 모드 (Baked Indirect mode)
 - 간접광만 구워서 미리 계산
 - 직사광과 그림자는 실시간으로 처리
- 섀도우마스크 (Shadowmask mode) - 기본값
 - 간접광을 위한 라이트 맵 이외에 그림자 맵(섀도우마스크 맵)을 추가로 구워서 사용
 - 실시간 그림자와 미리 구워진 그림자가 자연스럽게 합성됨)
- 감산 (Subtractive mode)
 - 간접광, 직사광, 그림자까지 하나의 라이트맵에 모두 구워버림
 - 실시간 그림자와 미리 구워진 그림자가 자연스럽게 합성되지 않음
 - 가장 오버헤드가 적음(성능이 좋음)

14.2 레벨 아트와 라이팅 설정(8)

라이트 컴포넌트의 모드

- 베이킹됨(Baked) 모드 : 빛을 미리 라이트 맵에 구워둠
 - 베이킹됨으로 설정된 라이트는 베이킹된 글로벌 일루미네이션에 반영
 - 라이트 효과를 미리 라이트맵에 굽는 설정이므로 아래와 같은 오브젝트에는 빛을 비추거나 그림자를 그릴 수 없음
 - 라이트를 굽는 시점에는 없던 게임 오브젝트
 - 게임 도중 움직일 수 있는 동적 게임 오브젝트
- 실시간(Realtime) 모드 : 실시간으로 빛을 연산
- 혼합(Mixed) 모드 : 실시간과 베이킹된 사이의 동작을 가짐
 - 동적 게임 오브젝트는 다음과 같은 빛 효과를 받음
 - 실시간 직사광
 - 실시간 그림자
 - 그림자 맵을 활용한 실시간 그림자
 - 정적 게임 오브젝트는 다음과 같은 빛 효과를 받음
 - 실시간 직사광
 - 라이트맵을 통해 미리 구워진 간접광
 - 미리 구워진 그림자
 - 그림자 맵을 활용한 실시간 그림자

14.2 레벨 아트와 라이팅 설정(9)

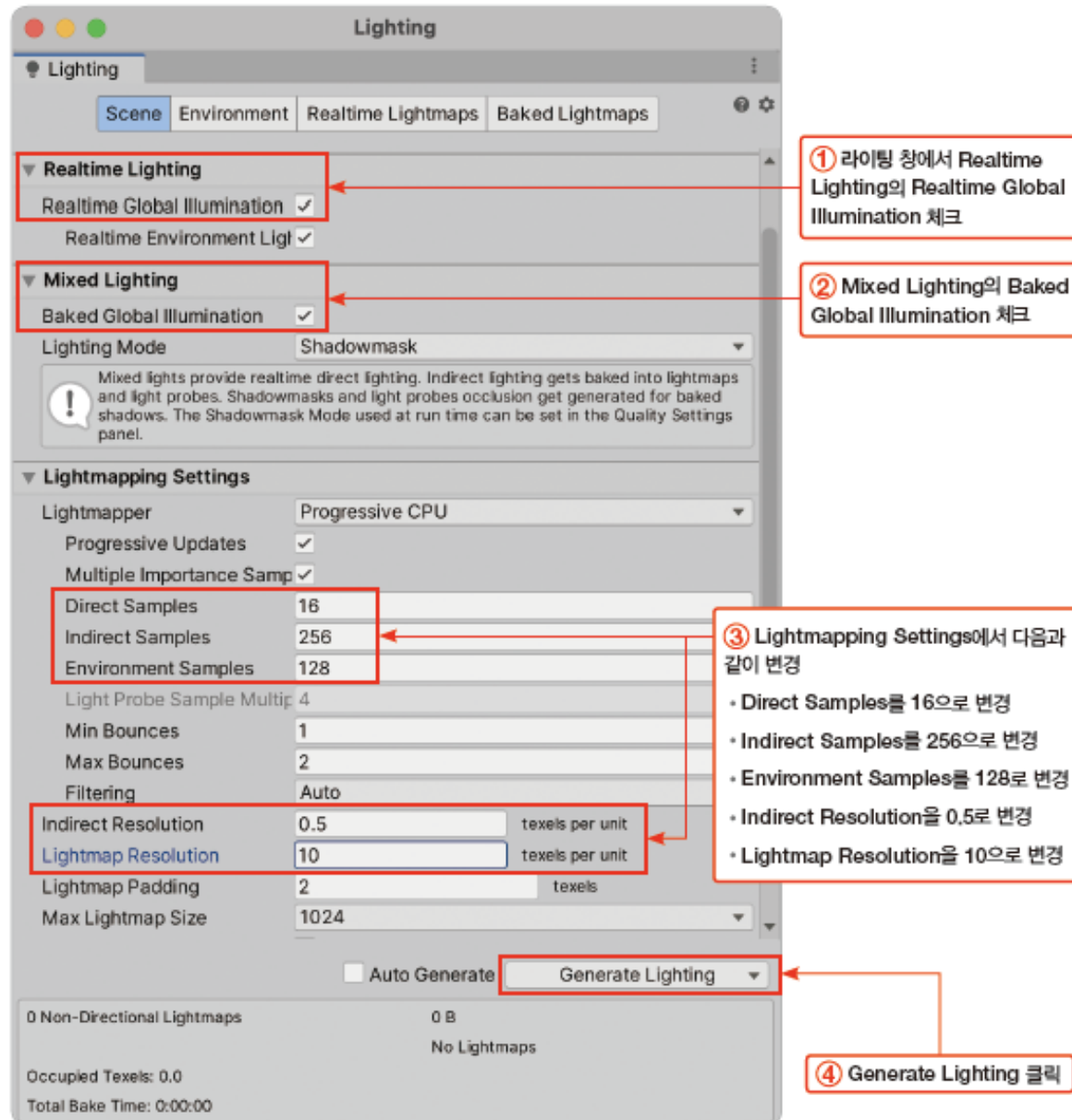
글로벌 일루미네이션 설정하기

- 실시간 글로벌 일루미네이션과 베이킹된 글로벌 일루미네이션을 모두 사용하여 라이트 맵 굽기

[과정 01] 라이트맵 굽기

- ① 라이팅 창에서 Realtime Lighting의 Realtime Global Illumination 체크
- ② Mixed Lighting의 Baked Global Illumination 체크
- ③ Lightmapping Settings에서 다음과 같이 변경
 - Direct Samples를 16으로 변경
 - Indirect Samples를 256으로 변경
 - Environment Samples를 128로 변경
 - Indirect Resolution을 0.5로 변경
 - Lightmap Resolution을 10으로 변경
- ④ Generate Lighting 클릭

14.2 레벨 아트와 라이팅 설정(10)



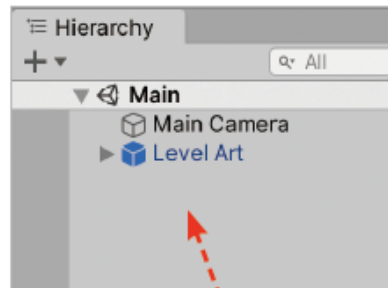
14.3 플레이어 캐릭터와 애니메이션 구성(1)

14.3.1 플레이어 캐릭터 준비하기

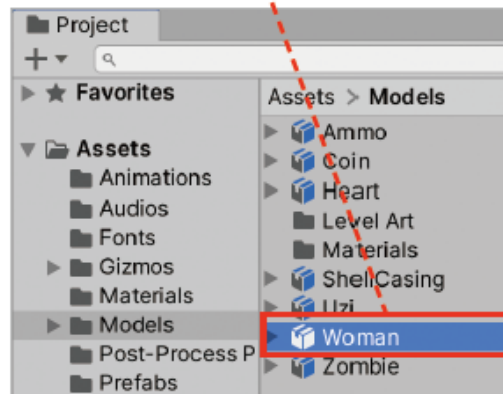
- 프로젝트 창 Models 폴더에 미리 준비한 3D 캐릭터 모델을 사용

[과정 01] 캐릭터 추가하기

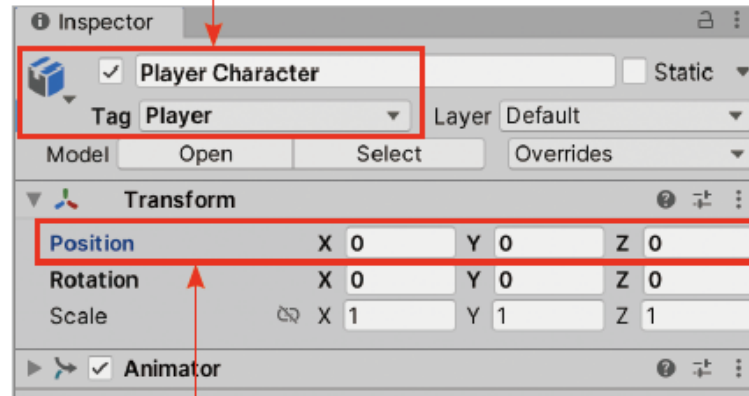
- ① Models 폴더에서 Woman 모델을 하이어라키 창으로 드래그&드롭
- ② 생성된 Woman 게임 오브젝트의 이름을 Player Character로, 태그를 Player로 변경
- ③ 위치를 (0, 0, 0)으로 변경



① Woman 모델을 하이어라키 창으로 드래그&드롭



② 이름을 Player Character로, 태그를 Player로 변경



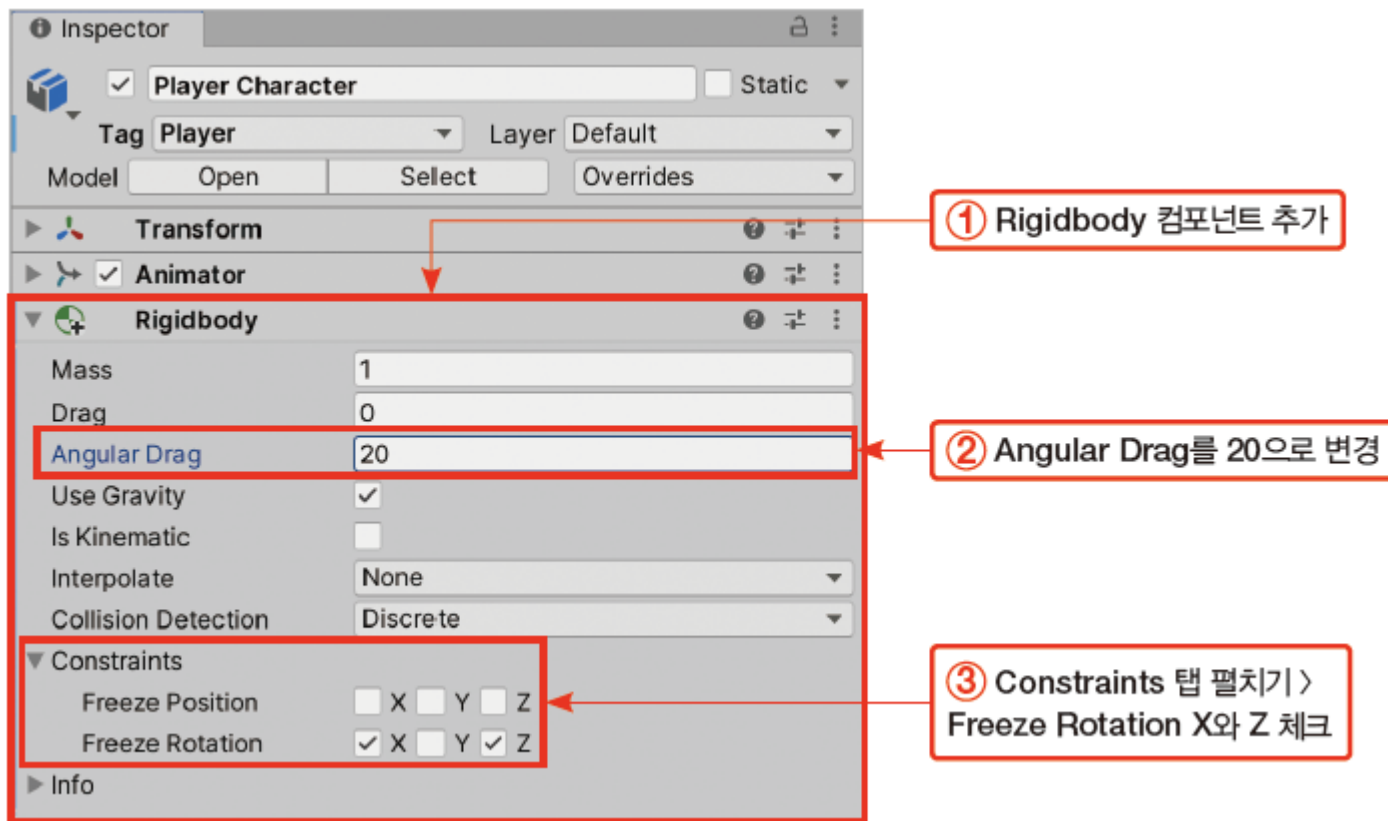
③ 위치를 (0, 0, 0)으로 변경

14.3 플레이어 캐릭터와 애니메이션 구성(2)

- 플레이어 캐릭터에 필요한 컴포넌트 추가

[과정 02] 캐릭터에 리지드바디 추가하기

- ① Rigidbody 컴포넌트 추가(Add Component > Physics > Rigidbody)
- ② Rigidbody 컴포넌트의 Angular Drag를 20으로 변경
- ③ Rigidbody 컴포넌트의 Constraints 탭 펼치기 > Freeze Rotation X와 Z 체크

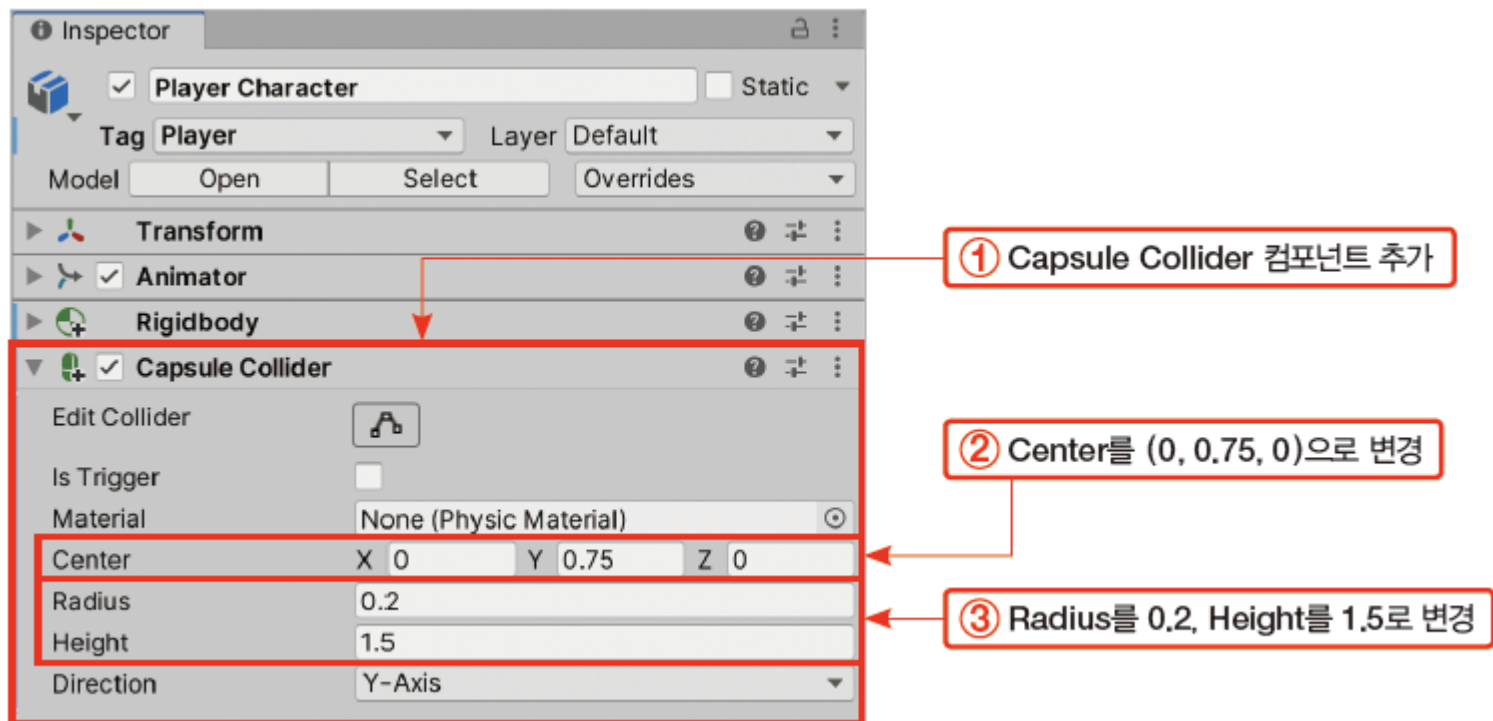


14.3 플레이어 캐릭터와 애니메이션 구성(3)

- 캐릭터에 콜라이더를 추가하여 물리적인 표면을 만들기

[과정 03] 캐릭터에 캡슐 콜라이더 추가하기

- ① Capsule Collider 컴포넌트 추가(Add Component > Physics > Capsule Collider)
- ② Capsule Collider 컴포넌트의 Center를 (0, 0.75, 0)으로 변경
- ③ Capsule Collider 컴포넌트의 Radius를 0.2, Height를 1.5로 변경

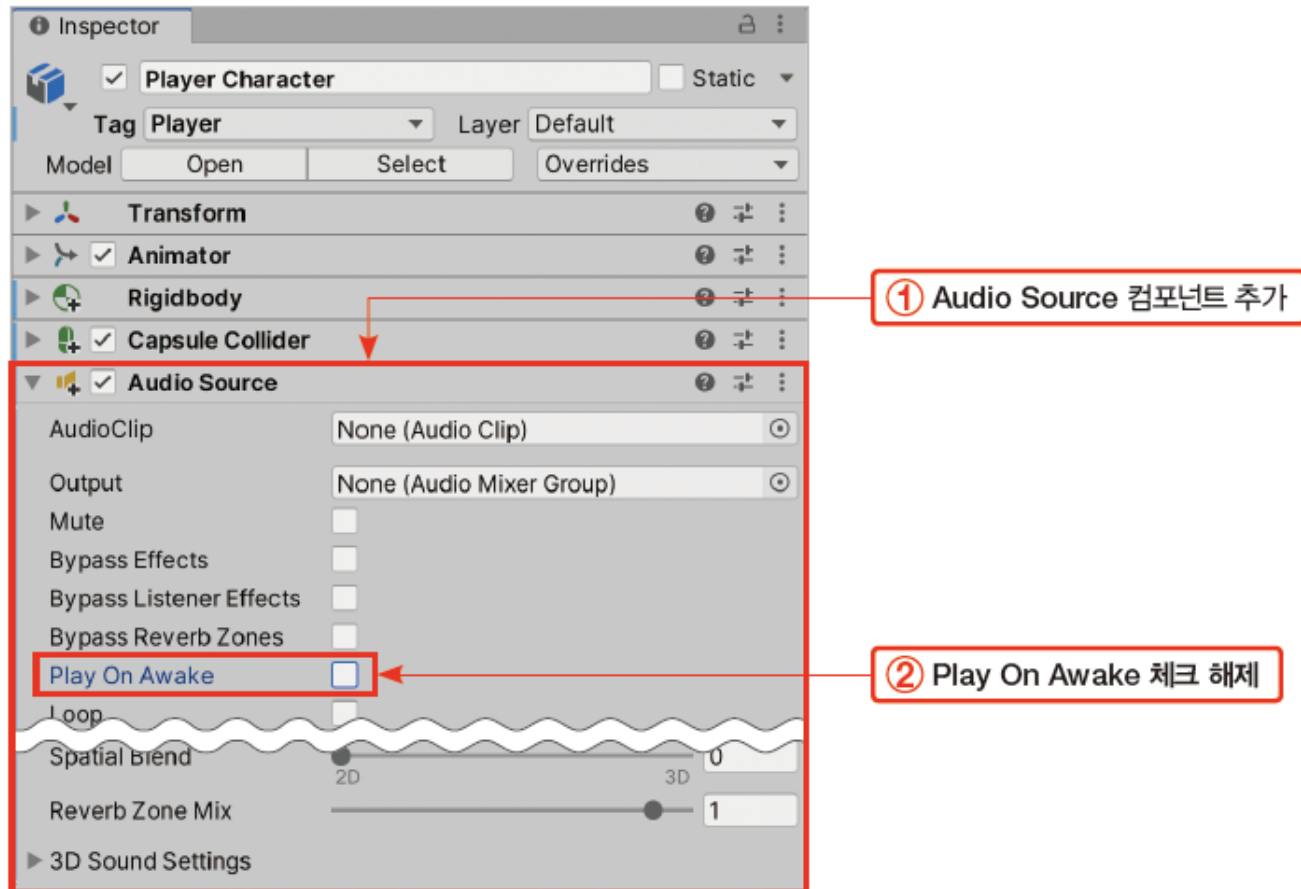


14.3 플레이어 캐릭터와 애니메이션 구성(4)

- 오디오 소스 추가

[과정 04] 캐릭터에 오디오 소스 추가하기

- ① Audio Source 컴포넌트 추가(Add Component > Audio > Audio Source)
- ② Audio Source 컴포넌트의 Play On Awake 체크 해제



14.3 플레이어 캐릭터와 애니메이션 구성(5)

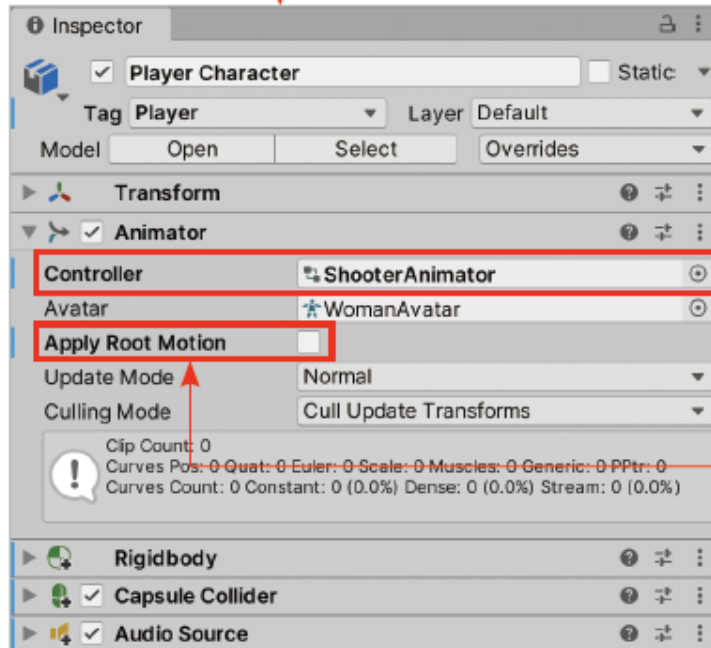
14.3.2 애니메이터 설정

- 플레이어 캐릭터에 사용할 ShooterAnimator 애니메이터 컨트롤러

[과정 01] 캐릭터의 애니메이터 설정하기

- ① 하이어라키 창에서 Player Character 게임 오브젝트 선택
- ② Animator 컴포넌트의 Controller 필드에 ShooterAnimator 애니메이터 컨트롤러 할당
(Controller 필드의 선택 버튼 클릭 > 선택 창에서 ShooterAnimator 더블 클릭)
- ③ Animator 컴포넌트의 Apply Root Motion 체크 해제

① Player Character 게임 오브젝트 선택



② Controller 필드에 ShooterAnimator
애니메이터 컨트롤러 할당

③ Apply Root Motion 체크 해제

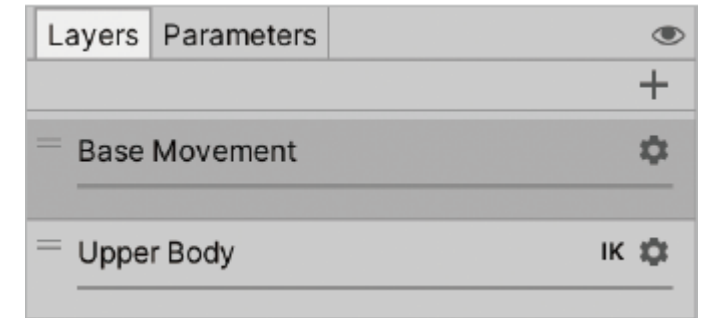
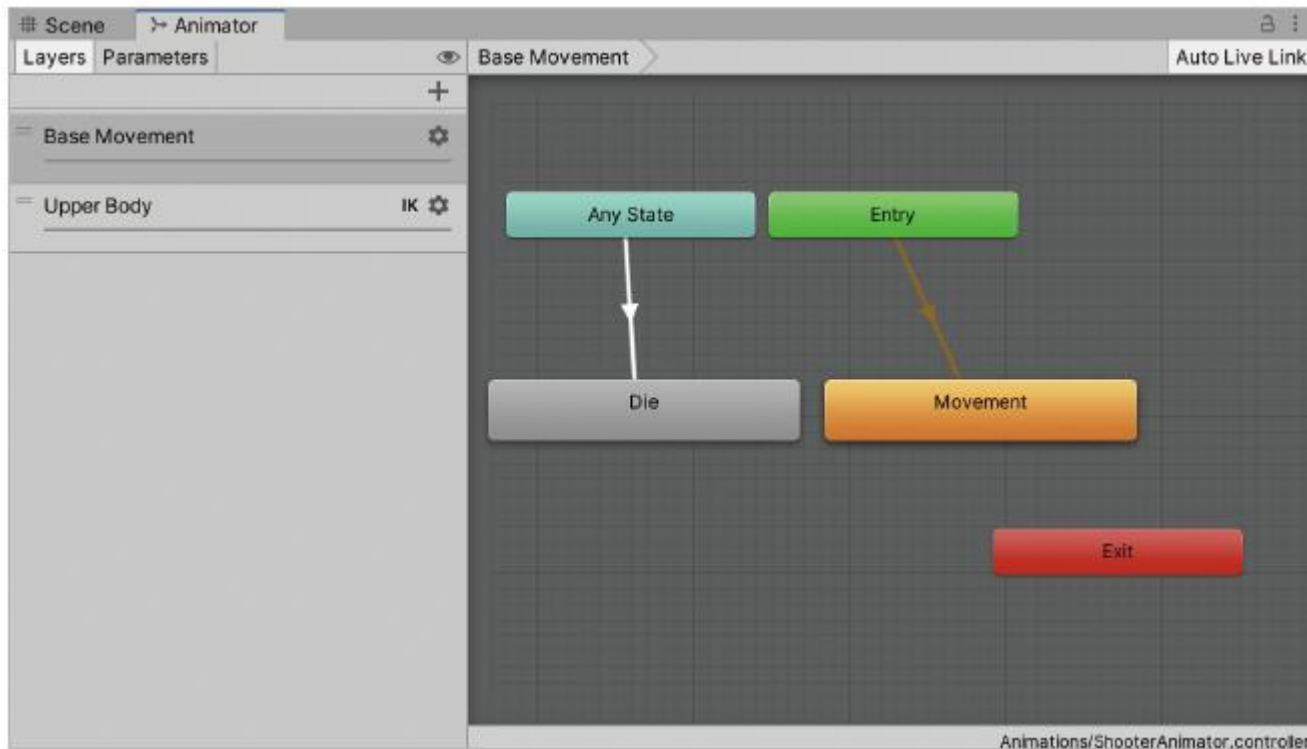
14.3 플레이어 캐릭터와 애니메이션 구성(6)

14.3.3 애니메이터 레이어

- ShooterAnimator 애니메이터 컨트롤러의 구성

[과정 01] ShooterAnimator 애니메이터 컨트롤러 표시하기

- ① 하이어라키 창에서 Player Character 게임 오브젝트 선택
- ② 애니메이터 창 띄우기(유니티 상단 메뉴의 Window > Animation > Animator)

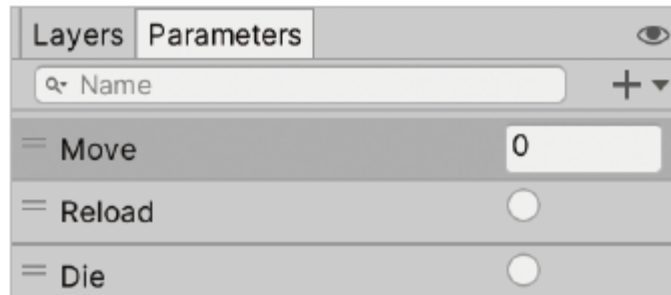


▶ 두 레이어

14.3 플레이어 캐릭터와 애니메이션 구성(7)

ShooterAnimator의 파라미터

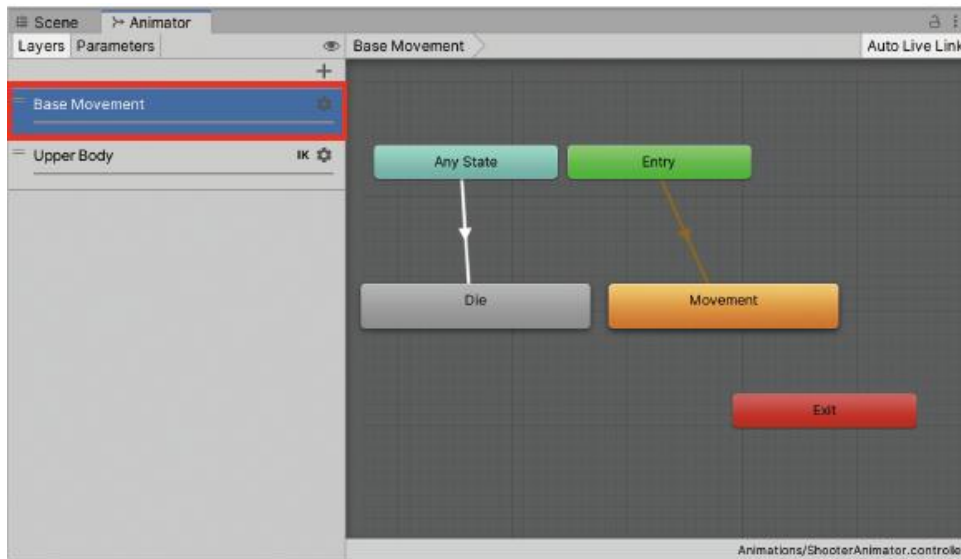
- 파라미터들은 애니메이터에 구성된 상태들 사이의 전이 조건으로 사용
- 각 파라미터에 할당될 값은 스크립트를 사용해 결정
 - Move : 앞뒤 움직임에 관한 입력값
 - Reload : 재장전을 알리는 트리거
 - Die : 사망을 알리는 트리거



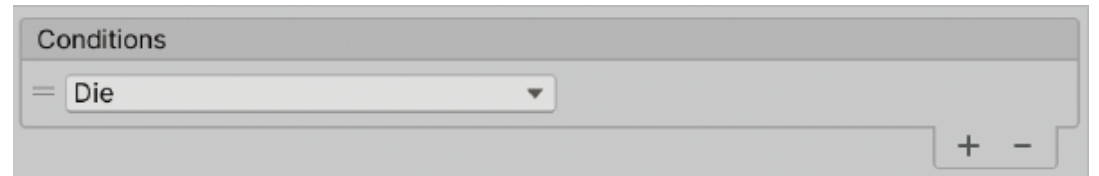
14.3 플레이어 캐릭터와 애니메이션 구성(8)

14.3.4 Base Movement 레이어

- Movement 상태는 Entry 상태와 이어져 있음
 - 애니메이터가 활성화될 때에는 Movement 상태가 가장 먼저 재생
 - Movement 상태는 캐릭터의 앞뒤 움직임에 관한 애니메이션을 재생
- Die 상태는 사망 애니메이션을 재생
 - Any State → Die 전이의 조건은 Die 트리거이며 Any State → Die 전이를 클릭하여 확인
 - 어떠한 경우라도 Die 트리거가 발동되면 Any State → Die 전이가 실행되어 사망 애니메이션이 재생



▶ Base Movement 레이어

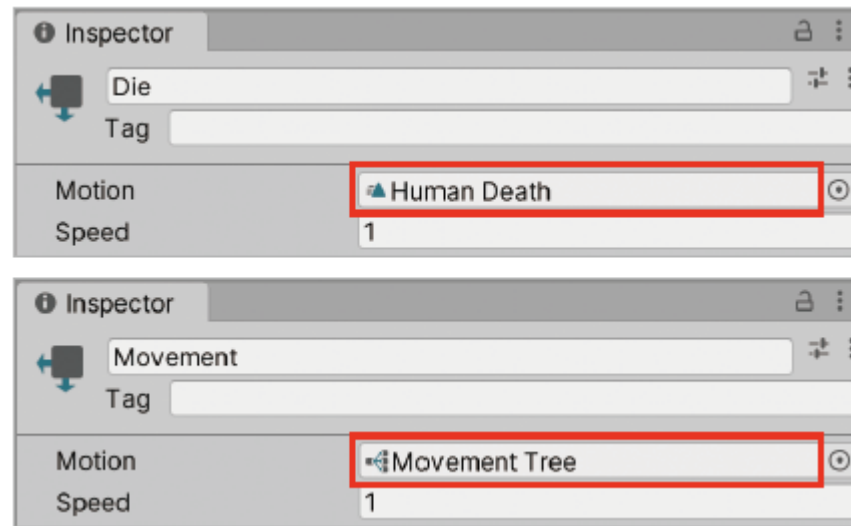


▶ Any State → Die 전이의 조건

14.3 플레이어 캐릭터와 애니메이션 구성(9)

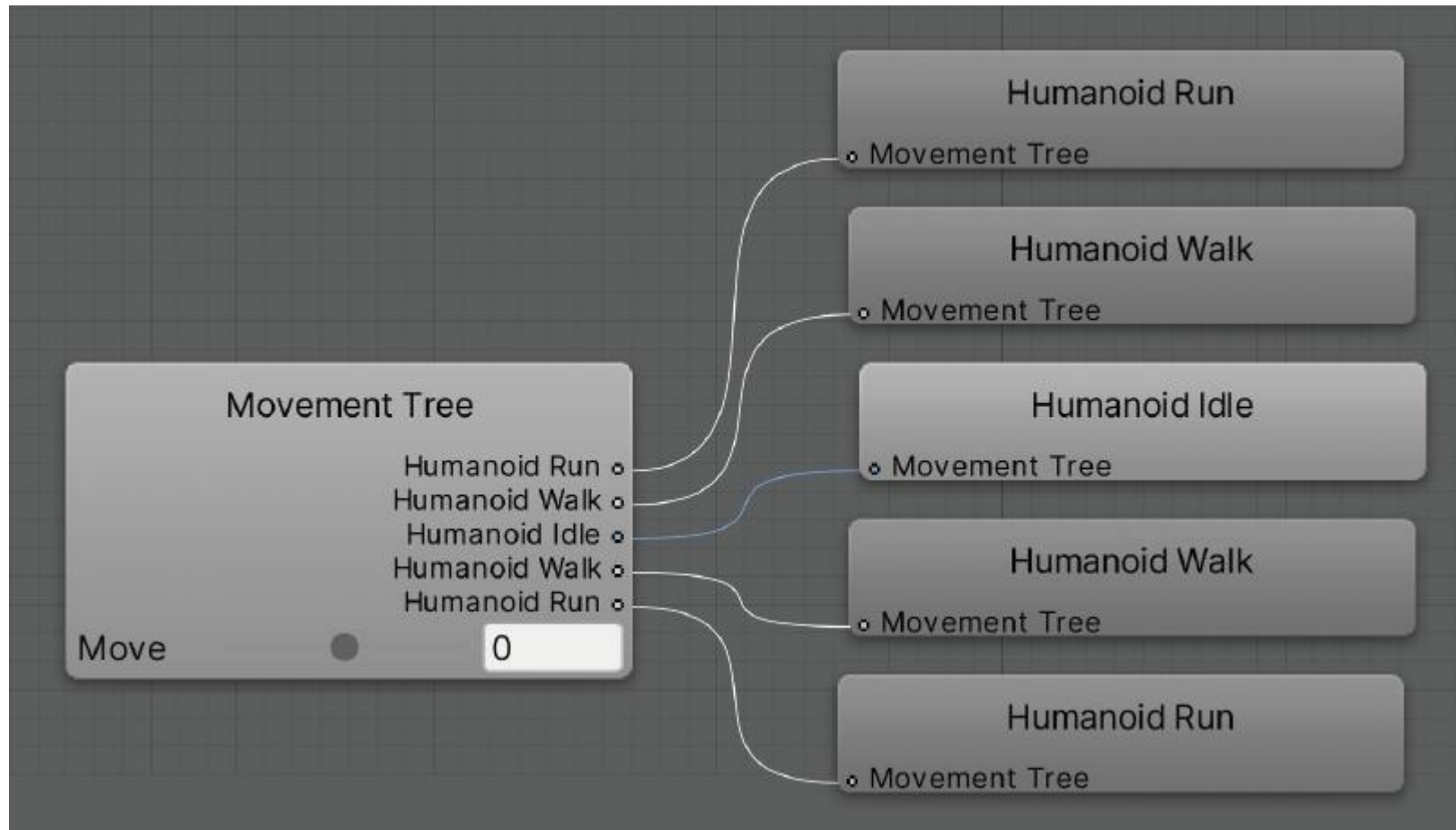
14.3.5 블렌드 트리

- 일반적으로 애니메이터의 상태에는 애니메이션 클립이 할당
- 평범한 애니메이션 클립이 아닌 특수한 종류의 모션(움직임을 나타내는 에셋)을 상태에 할당하는 것도 가능



▶ Die 상태에는 애니메이션 클립, Movement 상태에는 블렌드 트리가 할당됨

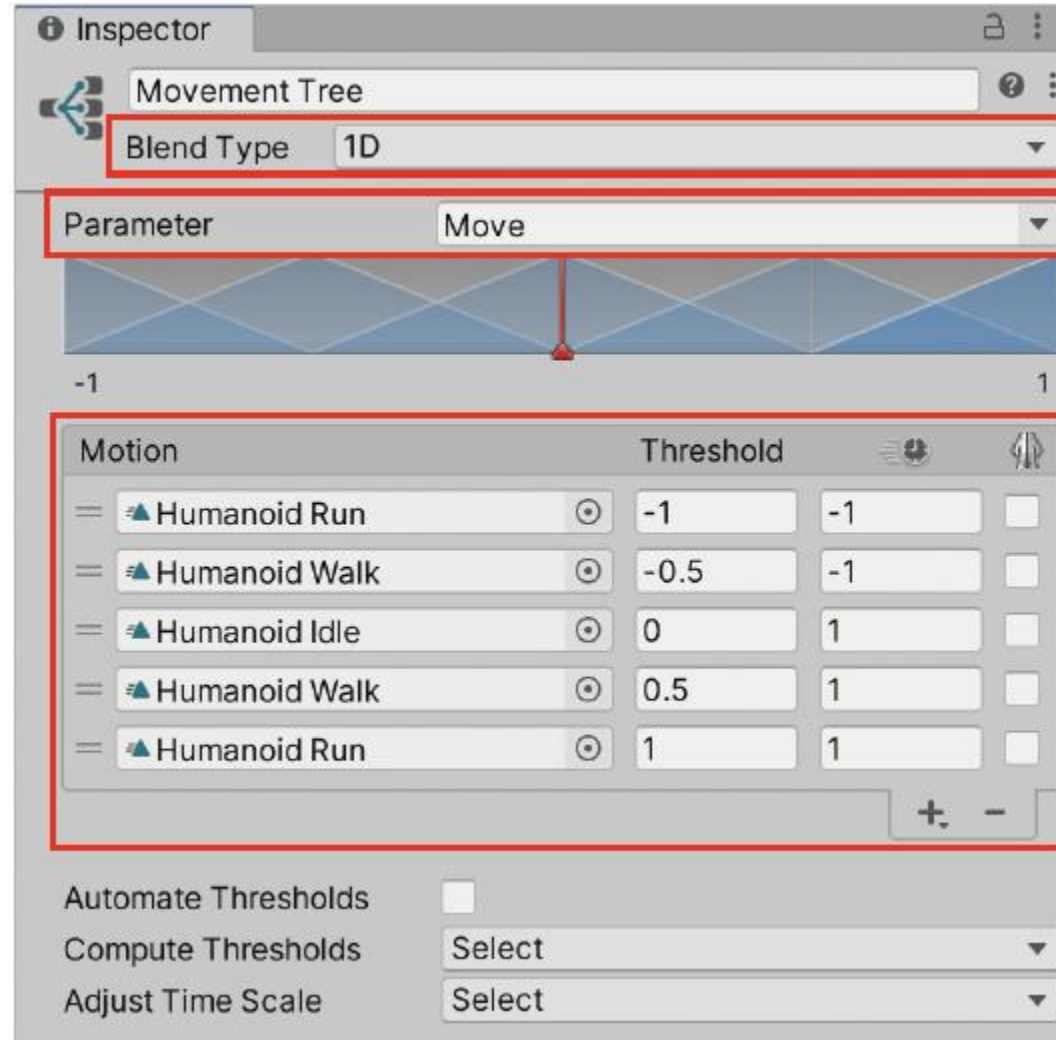
14.3 플레이어 캐릭터와 애니메이션 구성(10)



▶ Die 상태에는 애니메이션 클립, Movement 상태에는 블렌드 트리가 할당됨

14.3 플레이어 캐릭터와 애니메이션 구성(11)

- Movement Tree 노드를 클릭하고 인스펙터 창을 확인해보면 Movement Tree 노드의 상태가 다음과 같이 표시



▶ Movement 상태의 Blend Tree 모션 구성

14.3 플레이어 캐릭터와 애니메이션 구성(12)

- Movement Tree의 Motion 리스트에는 저자가 미리 애니메이션 클립 다섯 개를 할당
- 아래 애니메이션 클립이 Move 값에 따라 서로 섞이게 됨

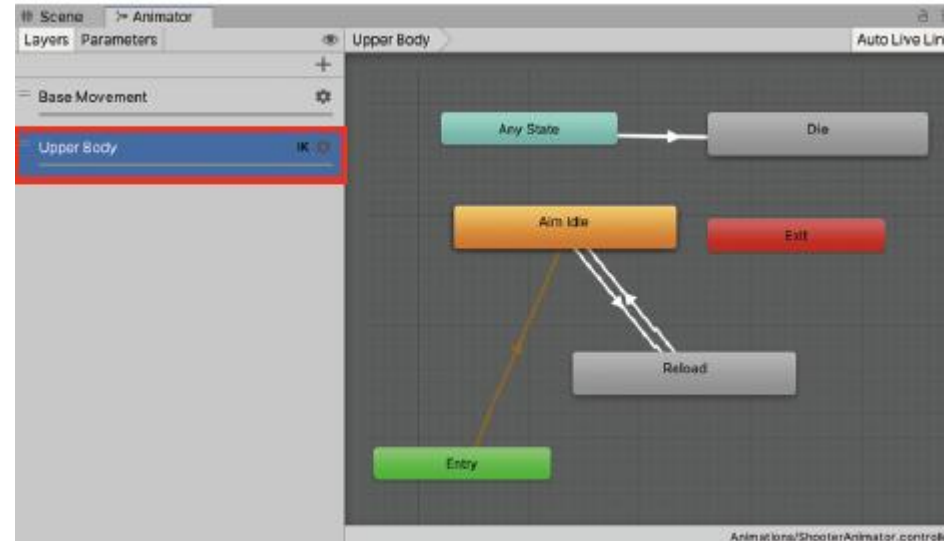
순서	애니메이션 클립(Motion)	임젯값(Threshold)	애니메이션 재생속도(Animation Speed)
1	Humanoid Run(뒤로 뛰기)	-1	-1
2	Humanoid Walk(뒤로 걷기)	-0.5	-1
3	Humanoid Idle(대기)	0	1
4	Humanoid Walk(걷기)	0.5	1
5	Humanoid Run(뛰기)	1	1

- 1번 클립과 2번 클립의 애니메이션 재생속도가 -1
 - 기존의 걷는 애니메이션 클립과 뛰는 애니메이션 클립을 거꾸로 재생하는 방식으로, 뒤로 걷는 애니메이션 클립과 뒤로 뛰는 애니메이션 클립을 대체
- 임젯값 - 자신의 애니메이션 클립이 100% 섞이는 지점
 - Move 값이 0.5와 1.0의 중간값인 0.75라면 4번 클립과 5번 클립이 반반씩 섞이게 됨
- 블렌드 트리를 사용하는 Movement 상태는 Move 값에 따라 '뒤로 뛰기 ⇄ 뒤로 걷기 ⇄ 대기 ⇄ 걷기 ⇄ 뛰기' 사이에서 애니메이션 클립을 자연스럽게 섞어 사용

14.3 플레이어 캐릭터와 애니메이션 구성(13)

14.3.6 Upper Body 레이어

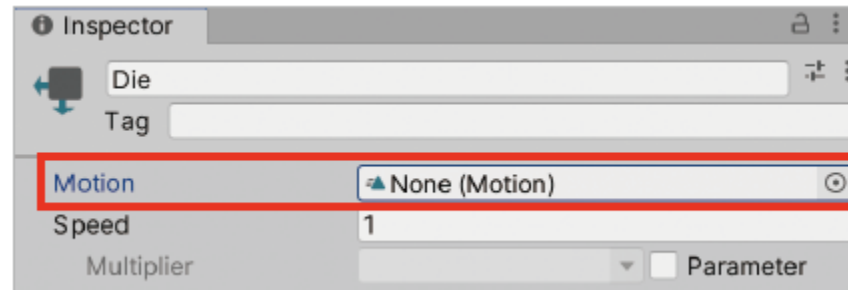
- 애니메이터 창의 Layers 탭에서 Upper Body 레이어를 클릭하여 Upper Body 레이어의 상태를 표시



▶ Upper Body 레이어의 구성

14.3 플레이어 캐릭터와 애니메이션 구성(14)

- Upper Body 레이어의 Die 상태는 Base Movement 레이어의 Die 상태와 같은 방식으로 동작
 - Die 트리거가 발동되는 순간 Any State → Die 전이에 의해 Die 상태가 재생
- 하지만 Base Movement 레이어의 Die 상태에서도 동시에 사망 애니메이션 클립을 재생
 - Upper Body 레이어의 Die 상태에는 애니메이션 클립을 할당하지 않음



▶ Motion 필드가 빈 Die 상태

14.3 플레이어 캐릭터와 애니메이션 구성(15)

- Upper Body 레이어의 Aim Idle 상태는 Upper Body 레이어에서 가장 먼저 재생
 - Aim Idle 상태 - 캐릭터가 총을 들고 조준하는 애니메이션 클립을 재생
 - Reload 상태 - 재장전 애니메이션 클립을 재생
- Aim Idle 상태와 Reload 상태는 양방향으로 전이가 연결
 - Aim Idle → Reload 전이는 Reload 트리거를 조건으로 사용
 - 반대로 Reload → Aim Idle 전이는 조건이 없음

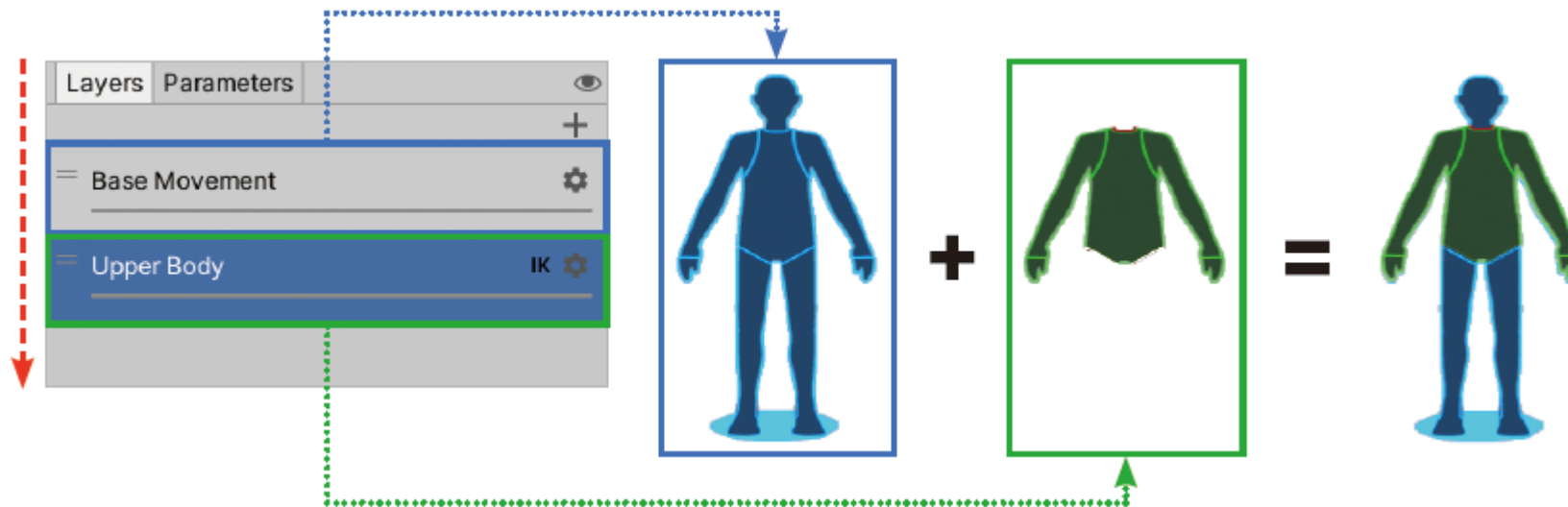


▶ Aim Idle 상태와 Reload 상태에 할당된 애니메이션 클립

14.3 플레이어 캐릭터와 애니메이션 구성(16)

14.3.7 애니메이터 레이어 나누기

- 애니메이터 컨트롤러에 레이어를 두 개 이상 만들면 각 레이어에서 재생하는 애니메이션은 위에서 아래 순서로 덮어쓰기 방식으로 적용
 - ShooterAnimator 애니메이터 컨트롤러에서는 Base Movement 레이어의 애니메이션이 먼저 캐릭터에 반영
 - Upper Body 레이어의 애니메이션이 Base Movement 레이어의 애니메이션을 덮어쓰기 하면서 캐릭터에 반영



▶ Base Movement 레이어와 Upper Body 레이어의 혼합

- 이렇게 레이어를 나눈 이유는 더 적은 애니메이션 클립으로 다양한 경우에 대응하기 위함임
- 뛰면서 재장전 → Base에서는 뛰는 애니메이션, Upper에서는 재장전

14.3 플레이어 캐릭터와 애니메이션 구성(17)

14.3.8 아바타 마스크

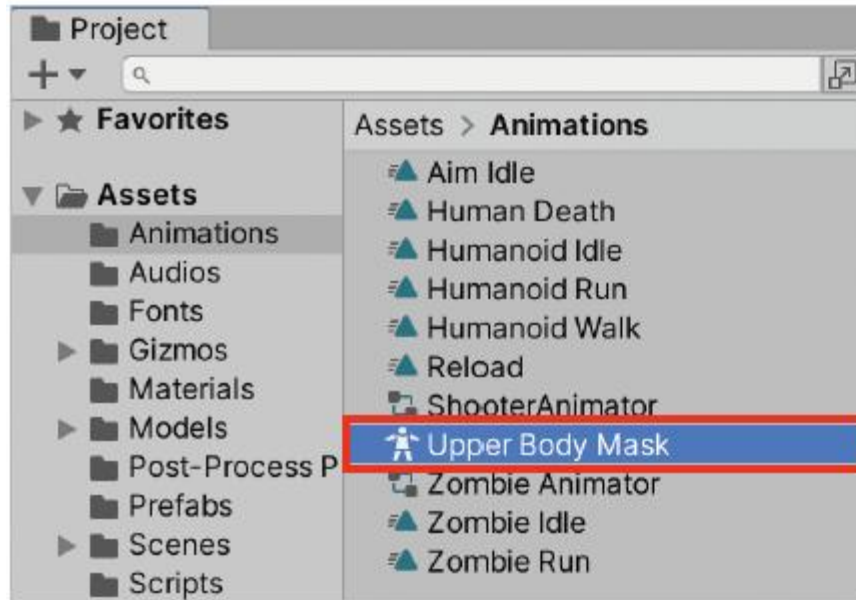
휴머노이드 릿(Humanoid Rig)

- 사람 형태의 3D 모델은 대부분 휴머노이드 Humanoid 타입으로 리깅(Rigging)
- 유니티에서 휴머노이드 타입으로 리깅된 3D 모델은 체형이 달라도 애니메이션 클립이 호환

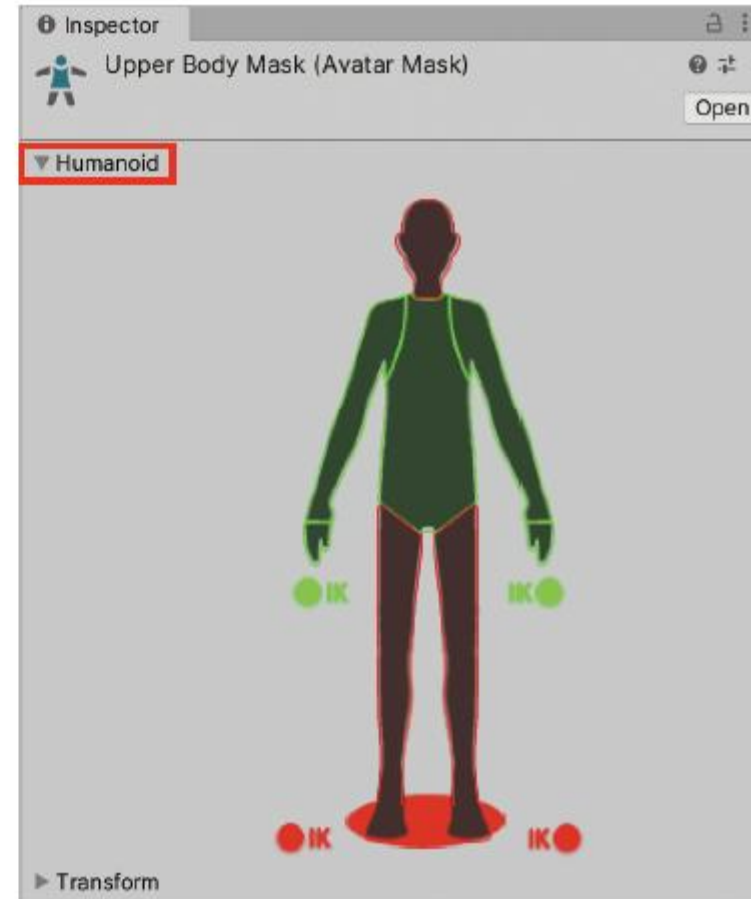
아바타 마스크

- 휴머노이드 타입의 3D 모델에서 특정 신체 부위에만 애니메이션을 적용
 - 아바타 마스크는 프로젝트 창에서 + > Avatar Mask로 생성
- Rigging : 3D모델의 골격과 움직임을 정의하는 조인트 계층 구조를 만드는 것

14.3 플레이어 캐릭터와 애니메이션 구성(18)



▶ Upper Body Mask 아바타 마스크

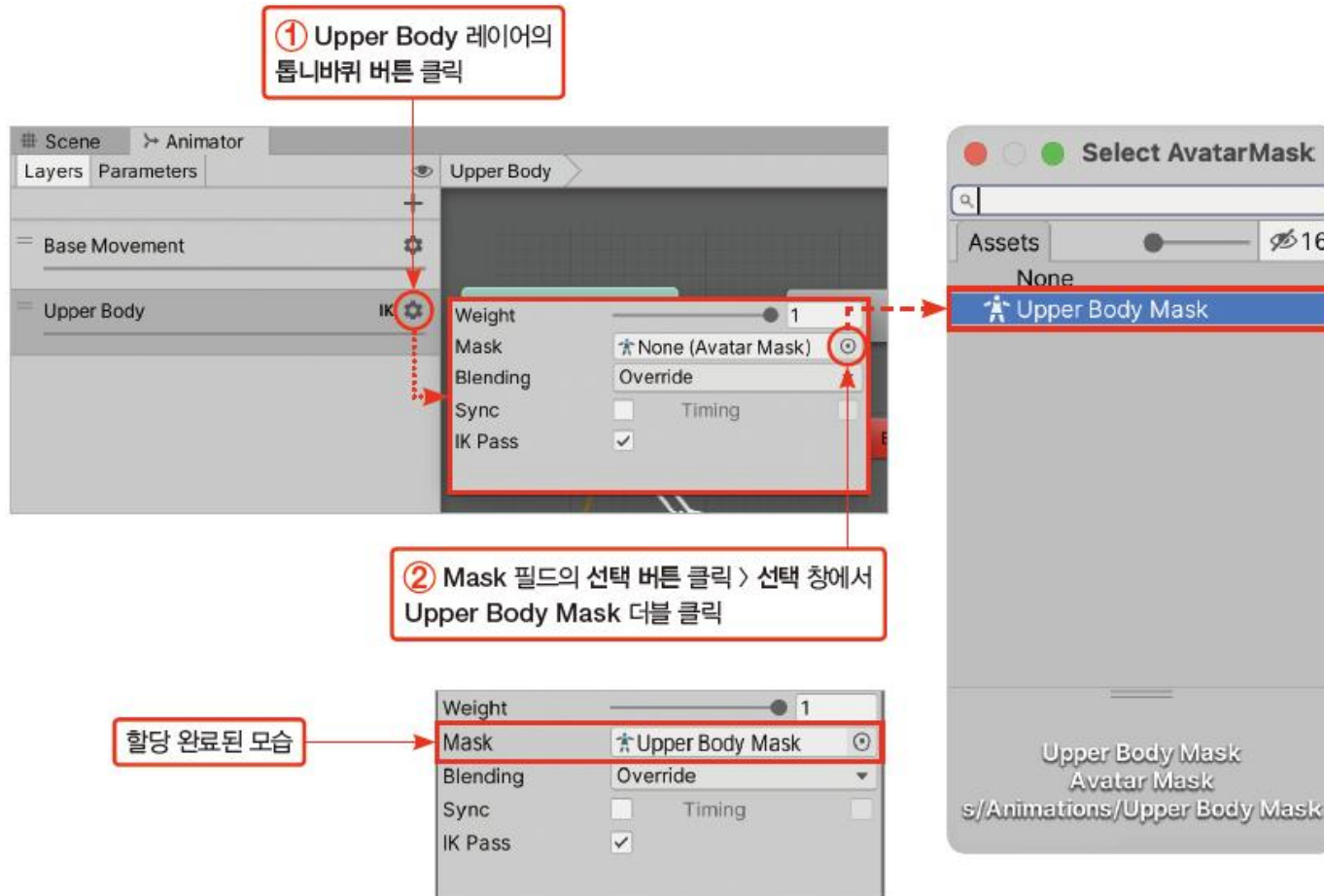


14.3 플레이어 캐릭터와 애니메이션 구성(19)

- Upper Body 레이어에 Upper Body Mask 마스크를 할당

[과정 01] Upper Body 레이어에 아바타 마스크 적용

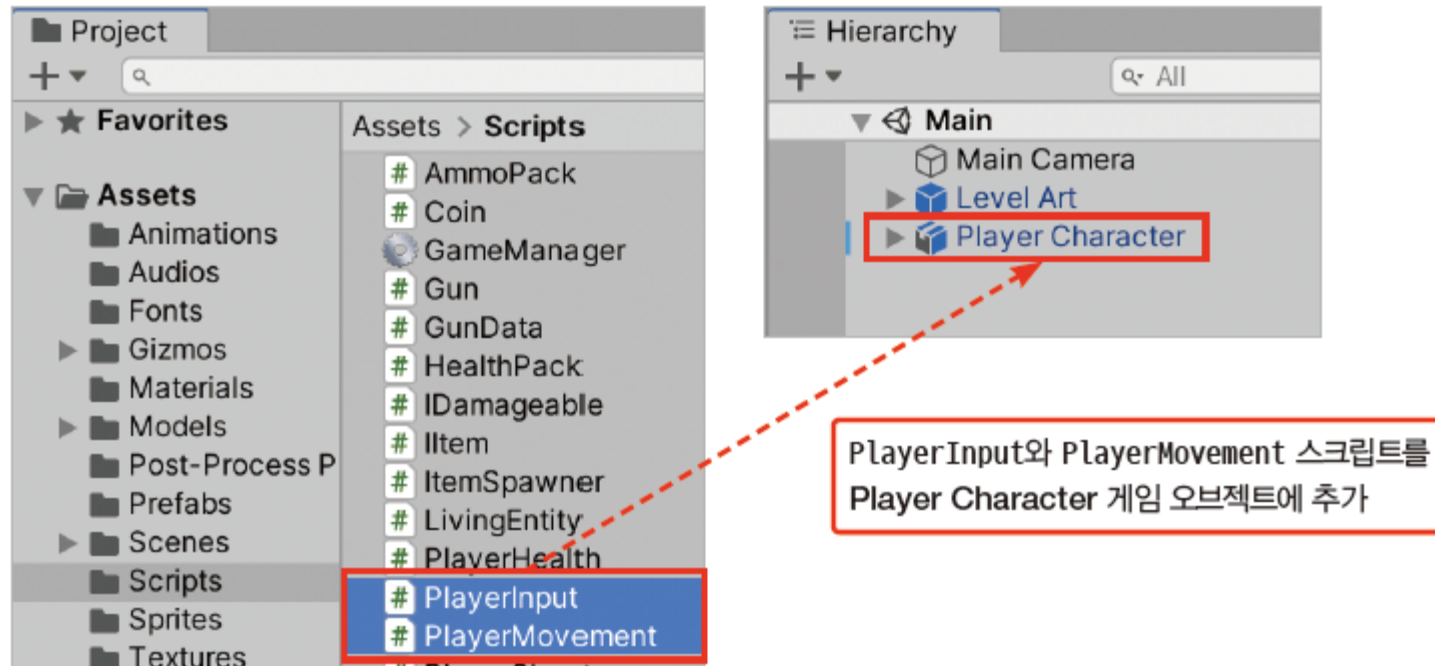
- ① Upper Body 레이어의 톱니바퀴 버튼 클릭
- ② Mask 필드의 선택 버튼 클릭 > 선택 창에서 Upper Body Mask 더블 클릭



14.4 캐릭터 이동 구현(1)

[과정 01] 캐릭터 이동을 위한 스크립트 추가

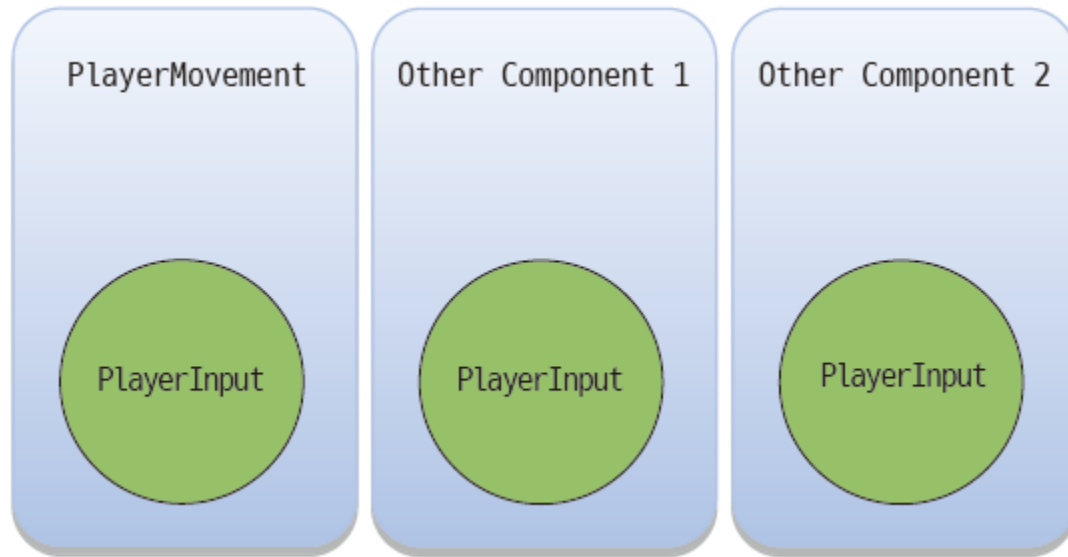
- ① 프로젝트 창에서 Scripts 폴더로 이동
- ② PlayerInput 스크립트를 하이어라키 창의 Player Character로 드래그&드롭
- ③ PlayerMovement 스크립트를 하이어라키 창의 Player Character로 드래그&드롭



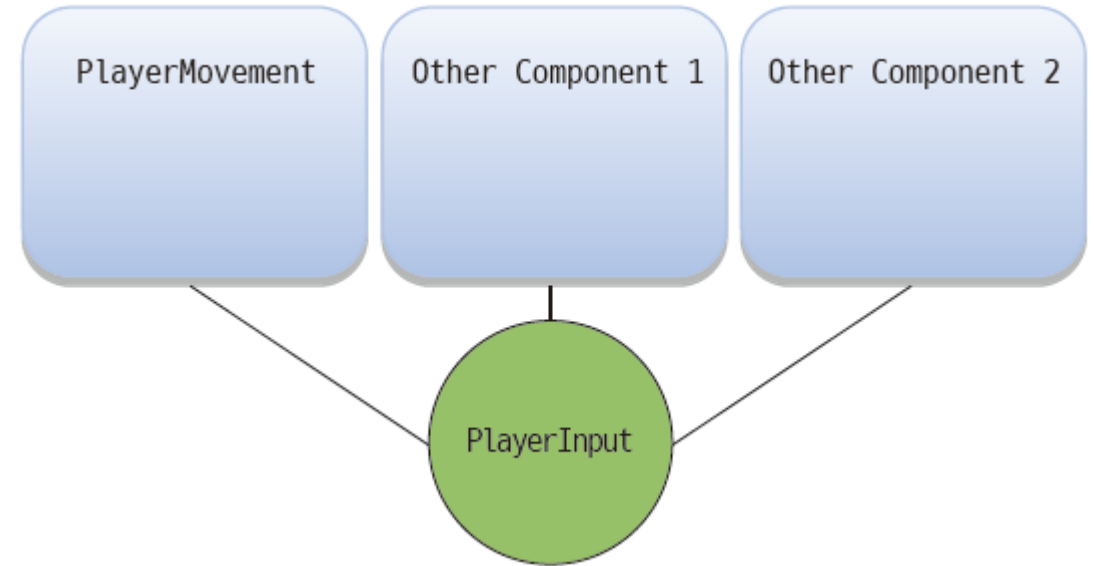
14.4 캐릭터 이동 구현(2)

14.4.1 입력과 액터 나누기

- 입력과 입력에 의한 행위(액터)를 나눔



플레이어 입력을 제각각 처리하는 경우



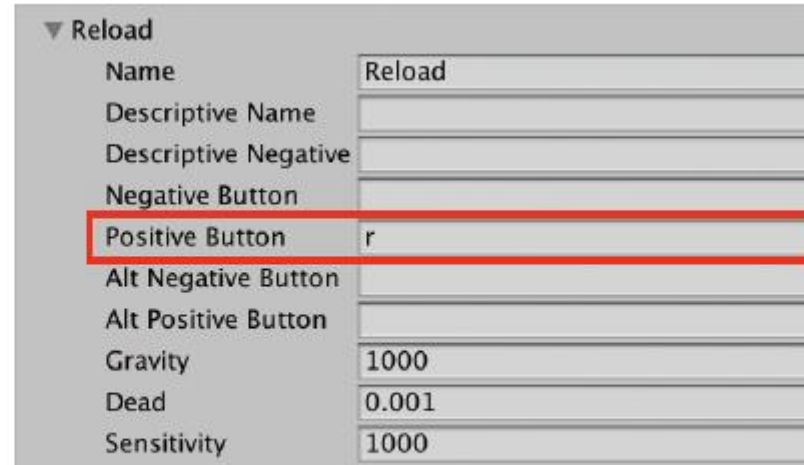
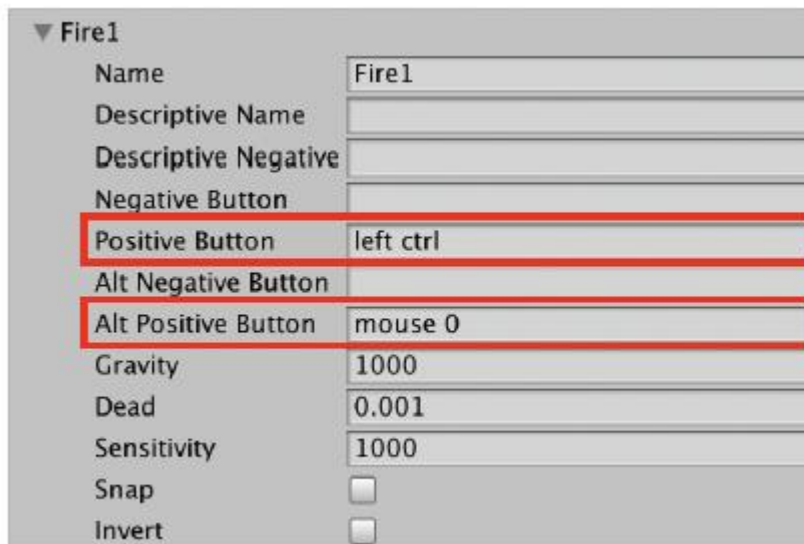
플레이어 입력을 별개의 컴포넌트로 만드는 경우

- 전체적인 코드양이 줄어들고,
필요할 때는 PlayerInput 스크립트만 수정

14.4 캐릭터 이동 구현(3)

14.4.2 PlayerInput 스크립트

- PlayerInput 스크립트는 플레이어의 입력을 감지하고 다른 컴포넌트에 알려줌
- string 변수 - 사용할 입력축과 버튼의 이름
 - moveAxisName : 앞뒤 움직임에 사용할 입력축(Vertical)
 - rotateAxisName : 좌우 회전에 사용할 입력축(Horizontal)
 - fireButtonName : 발사에 사용할 입력 버튼(Fire1)
 - reloadButtonName : 재장전에 사용할 입력 버튼(Reload)
- 버튼은 버튼을 눌렀을 때는 true, 버튼을 누르지 않았을 때는 false가 반환



▶ 입력 매니저에서 확인한 Fire1과 Reload

14.4 캐릭터 이동 구현(4)

- 감지된 입력값을 나타낼 프로퍼티
 - move : 감지된 움직임 입력. -1.0일 때 후진, +1.0일 때 전진
 - rotate : 감지된 회전 입력. -1.0일 때 왼쪽으로 회전, +1.0일 때 오른쪽으로 회전
 - fire : 감지된 발사 입력. true일 때 탄알 발사
 - reload : 감지된 재장전 입력. true일 때 탄알 재장전

```
public float move { get; private set; }  
public float rotate { get; private set; }  
public bool fire { get; private set; }  
public bool reload { get; private set; }
```

14.4 캐릭터 이동 구현(5)

14.4.3 프로퍼티

- 프로퍼티는 변수값을 읽거나 쓰는 과정에서 유연한 처리를 삽입할 수 있는 클래스 멤버
- 프로퍼티는 변수처럼 보이지만 변수가 아닌 특수한 형태의 메서드
- 디스크의 용량을 기록하는 VolumeInfo 클래스를 작성한다고 가정
 - VolumeInfo 클래스는 바이트byte 단위로 디스크의 용량을 계산하여 기록
 - 필요 시 킬로바이트kilobyte 등의 다른 단위로 디스크의 용량을 변환하여 제공

```
public class VolumeInfo {  
    public float megaBytes  
    {  
        get { return m_bytes * 0.000001f; }  
  
        set  
        {  
            if (value <= 0) {  
                m_bytes = 0;  
            } else {  
                m_bytes = value * 1000000f;  
            }  
        }  
    }  
}
```

(이하 코드 생략)

14.4 캐릭터 이동 구현(6)

- VolumeInfo 오브젝트를 생성하여 새로운 디스크 용량을 기록하고 다양한 단위로 출력한다고 가정 (간략화한 예시이며 실제로는 이런 식으로 용량을 기록하지 않음)
 - 프로퍼티에 새로운 값을 할당할 경우 프로퍼티의 set 접근자가 실행
 - 예시 코드에서 info.bytes = 1000000;이 실행되면 bytes의 set이 실행되고 할당 연산자 =을 통해 전달받은 값이 set 내부의 value 키워드로 전달
 - bytes의 set 접근자는 value 값이 0 이상인 경우 m_bytes에 value 값을 할당

```
set
{
    if (value <= 0) {
        m_bytes = 0;
    } else {
        m_bytes = value;
    }
}
```

- Debug.Log(info.kiloBytes);처럼 프로퍼티값을 가져오려는 경우 프로퍼티의 get 접근자가 실행
 - kiloBytes 프로퍼티의 get 접근자는 m_bytes의 값에 0.001을 곱한 값을 반환

```
get { return m_bytes * 0.001f; }
```

14.4 캐릭터 이동 구현(7)

- info.megaBytes = 4;가 실행되면 실제로는 megaBytes의 set 접근자가 실행
 - info 내부에서는 m_bytes에 4 * 1000000을 할당하는 처리가 실행
 - info.megaBytes = 4;를 실행한 다음 info.bytes를 이용해 m_bytes의 값을 출력하면 4000000이 출력

```
set
{
    if (value <= 0) {
        m_bytes = 0;
    } else {
        m_bytes = value * 1000000f;
    }
}
```

14.4 캐릭터 이동 구현(8)

자동 변환

- VolumeInfo 예시에서 바이트 단위로 저장된 용량을 여러 단위의 값으로 즉시 출력
 - 이처럼 프로퍼티를 사용하면 어떤 값을 다른 맥락에 맞춰 제공

```
public float health = 100;
```

```
public bool dead {  
    get  
    {  
        if (health <= 0) {  
            return true;  
        }  
        return false;  
    }  
}
```

14.4 캐릭터 이동 구현(9)

안전성 증가

- 프로퍼티는 데이터를 안전하게 다루는 데 도움
 - 프로퍼티를 이용해 값을 읽고 할당하는 처리에 '필터'를 추가하면 외부에서 잘못된 값을 할당했을 때 내부에서 걸러냄

```
public float bytes
{
    get { return m_bytes; }

    set
    {
        if (value <= 0) {
            m_bytes = 0;
        } else {
            m_bytes = value;
        }
    }
}
```

```
info.bytes = -1000;
```

- 디스크가 0보다 작은 용량을 가진다는 것은 불가능 (즉, -1000은 잘못된 값)
- bytes의 set 접근자는 0보다 작은 값이 할당된 경우 해당 값을 사용하지 않고 m_bytes에 0을 할당

14.4 캐릭터 이동 구현(10)

접근자 개별 설정

- 프로퍼티의 get과 set 접근자는 접근 권한을 각각 다르게 부여
 - 예를 들어 VolumeInfo의 megaBytes의 set만 private로 선언

```
public float megaBytes
{
    get { return m_bytes * 0.000001f; }

    private set
    {
        if (value <= 0) {
            m_bytes = 0;
        } else {
            m_bytes = value * 1000000f;
        }
    }
}
```

- 프로퍼티의 get과 set 중 하나만 사용

```
public float megaBytes
{
    get { return m_bytes * 0.000001f; }
}
```

14.4 캐릭터 이동 구현(11)

자동 구현 프로퍼티

- PlayerInput 클래스에 선언된 move, rotate, fire, reload는 자동 구현 프로퍼티(Auto Implemented Properties)
- 자동 구현 프로퍼티는 get과 set의 접근 권한을 분리하는 것 이외의 처리가 필요하지 않을 때 사용

```
public float move { get; private set; }
```

- PlayerInput에서 자동 구현 프로퍼티 move는 아래 코드를 간결하게 자동 생성

```
public float move {  
    get { return m_move; }  
    private set { m_move = value; }  
}  
  
private float m_move;
```

14.4 캐릭터 이동 구현(12)

14.4.4 PlayerInput의 Update() 메서드

- Update() 메서드에서는 입력을 감지하고 감지된 입력값을 프로퍼티에 할당
- 게임 매니저는 싱글톤으로 구현되어 있으며, 변수 isGameOver를 이용해 게임오버 상태를 알려줌
 - 게임오버 상태에서는 사용자 입력값을 강제로 초기화하고 Update() 메서드를 종료

```
if (GameManager.instance != null && GameManager.instance.isGameOver)
{
    move = 0;
    rotate = 0;
    fire = false;
    reload = false;
    return;
}
```

- 게임오버가 아니거나 게임오버 상태를 표시하는 게임 매니저가 씬에 존재하지 않는 경우 이어지는 입력 감지 코드가 실행

```
move = Input.GetAxis(moveAxisName);
rotate = Input.GetAxis(rotateAxisName);
fire = Input.GetButton(fireButtonName);
reload = Input.GetButtonDown(reloadButtonName);
```

14.4 캐릭터 이동 구현(13)

14.4.5 PlayerMovement 스크립트

- PlayerMovement 스크립트를 더블 클릭해서 열고 편집 – 비어있는 메서드 완성
- 선언된 변수
 - moveSpeed - 앞뒤 이동 속도
 - rotateSpeed - 좌우 회전 속도
 - playerInput - 플레이어 입력을 전달하는 PlayerInput 컴포넌트를 할당
 - playerRigidbody - 플레이어 캐릭터의 리지드바디 컴포넌트가 할당
 - playerAnimator- 플레이어 캐릭터의 애니메이터 컴포넌트가 할당

```
public float moveSpeed = 5f;  
public float rotateSpeed = 180f;
```

```
private PlayerInput playerInput;  
private Rigidbody playerRigidbody;  
private Animator playerAnimator;
```

14.4 캐릭터 이동 구현(14)

◦ 14.4.6 PlayerMovement의 Start() 메서드

- PlayerMovement 스크립트의 Start() 메서드는 사용할 컴포넌트를 Player Character 게임오브젝트에서 GetComponent() 메서드로 찾아 변수에 할당

[과정 01] PlayerMovement의 Start () 메서드 완성하기

① PlayerMovement 스크립트의 Start() 메서드를 다음과 같이 완성

```
private void Start() {  
    // 사용할 컴포넌트의 참조 가져오기  
    playerInput = GetComponent<PlayerInput>();  
    playerRigidbody = GetComponent<Rigidbody>();  
    playerAnimator = GetComponent<Animator>();  
}
```

14.4 캐릭터 이동 구현(15)

14.4.7 PlayerMovement의 FixedUpdate() 메서드

- FixedUpdate()는 Update()처럼 유니티 이벤트 메서드로서 주기적으로 자동 실행
 - 화면 갱신 주기에 맞춰 실행되는 Update()와 달리 FixedUpdate()는 물리 정보 갱신 주기(기본값 0.02초)에 맞춰 실행

[과정 01] PlayerMovement의 FixedUpdate () 메서드 완성하기

① PlayerMovement 스크립트의 FixedUpdate() 메서드를 다음과 같이 완성

```
private void FixedUpdate() {  
    // 회전 실행  
    Rotate();  
    // 움직임 실행  
    Move();  
  
    // 입력값에 따라 애니메이터의 Move 파라미터값 변경  
    playerAnimator.SetFloat("Move", playerInput.move);  
}
```

14.4 캐릭터 이동 구현(16)

14.4.8 PlayerMovement의 Move() 메서드

- Move() 메서드는 감지된 플레이어 입력에 플레이어 캐릭터를 실제로 이동

[과정 01] PlayerMovement의 Move () 메서드 완성하기

① PlayerMovement 스크립트의 Move() 메서드를 다음과 같이 완성

```
private void Move() {  
    // 상대적으로 이동할 거리 계산  
    Vector3 moveDistance =  
        playerInput.move * transform.forward * moveSpeed * Time.deltaTime;  
    // 리지드바디를 이용해 게임 오브젝트 위치 변경  
    playerRigidbody.MovePosition(playerRigidbody.position + moveDistance);  
}
```

14.4 캐릭터 이동 구현(17)

- Vector3 moveDistance는 한 프레임(Time.deltaTime) 동안 현재 위치에서 상대적으로 더 이동할 거리와 방향

```
Vector3 moveDistance =  
    playerInput.move * transform.forward * moveSpeed * Time.deltaTime;
```

- 플레이어 캐릭터가 앞쪽 방향으로 이동한다고 가정했을 때 이동할 거리와 방향은 앞쪽 방향 * 속력 * 시간

```
transform.forward * moveSpeed * Time.deltaTime;
```

- 계산된 이동거리에 감지된 사용자 입력값 playerInput.move를 곱함

```
playerInput.move * transform.forward * moveSpeed * Time.deltaTime;
```

- 이동거리와 방향을 계산한 다음, 리지드바디의 MovePosition() 메서드로 게임 오브젝트의 위치를 변경
 - 트랜스폼의 위치값을 직접 변경하면 물리 처리를 무시하고 위치를 덮어쓰기 때문에, 막힌 벽 등을 무시하고 벽 반대쪽으로 이동할 수도 있음

```
playerRigidbody.MovePosition(playerRigidbody.position + moveDistance);
```

14.4 캐릭터 이동 구현(18)

14.4.9 PlayerMovement의 Rotate() 메서드

- Rotate() 메서드는 플레이어 회전에 관한 입력값 playerInput.rotate를 사용하여 캐릭터를 회전

[과정 01] PlayerMovement의 Rotate () 메서드 완성하기

① PlayerMovement 스크립트의 Rotate() 메서드를 다음과 같이 완성

```
private void Rotate() {  
    // 상대적으로 회전할 수치 계산  
    float turn = playerInput.rotate * rotateSpeed * Time.deltaTime;  
    // 리지드바디를 이용해 게임 오브젝트 회전 변경  
    playerRigidbody.rotation =  
        playerRigidbody.rotation * Quaternion.Euler(0, turn, 0f);  
}
```

14.4 캐릭터 이동 구현(19)

- turn은 사용자 입력에 따라 한 프레임 동안 회전할 각도를 저장하는 변수
 - turn의 값은 사용자 입력 * 회전 속도 * 시간으로 계산

```
playerInput.rotate * rotateSpeed * Time.deltaTime
```

- 현재 회전에서 turn만큼 Y축 기준으로 더 회전
 - 쿼터니언 곱으로 구현

```
playerRigidbody.rotation =  
    playerRigidbody.rotation * Quaternion.Euler(0, turn, 0f);
```

14.4 캐릭터 이동 구현(20)

14.4.10 PlayerMovement 스크립트의 전체 코드

```
using UnityEngine;

// 사용자 입력에 따라 플레이어 캐릭터를 움직이는 스크립트
public class PlayerMovement : MonoBehaviour {
    public float moveSpeed = 5f; // 앞뒤 움직임의 속도
    public float rotateSpeed = 180f; // 좌우 회전 속도

    private PlayerInput playerInput; // 플레이어 입력을 알려주는 컴포넌트
    private Rigidbody playerRigidbody; // 플레이어 캐릭터의 리지드바디
    private Animator playerAnimator; // 플레이어 캐릭터의 애니메이터

    private void Start() {
        // 사용할 컴포넌트의 참조 가져오기
        playerInput = GetComponent<PlayerInput>();
        playerRigidbody = GetComponent<Rigidbody>();
        playerAnimator = GetComponent<Animator>();
    }
```

▶ 다음 쪽에 이어짐

14.4 캐릭터 이동 구현(21)

◀ 앞쪽에 이어

```
// FixedUpdate는 물리 갱신 주기에 맞춰 실행됨
private void FixedUpdate() {
    // 회전 실행
    Rotate();
    // 움직임 실행
    Move();

    // 입력값에 따라 애니메이터의 Move 파라미터값 변경
    playerAnimator.SetFloat("Move", playerInput.move);
}

// 입력값에 따라 캐릭터를 앞뒤로 움직임
private void Move() {
    // 상대적으로 이동할 거리 계산
    Vector3 moveDistance =
        playerInput.move * transform.forward * moveSpeed * Time.deltaTime;
    // 리지드바디를 이용해 게임 오브젝트 위치 변경
    playerRigidbody.MovePosition(playerRigidbody.position + moveDistance);
}
```

▶ 다음 쪽에 이어짐

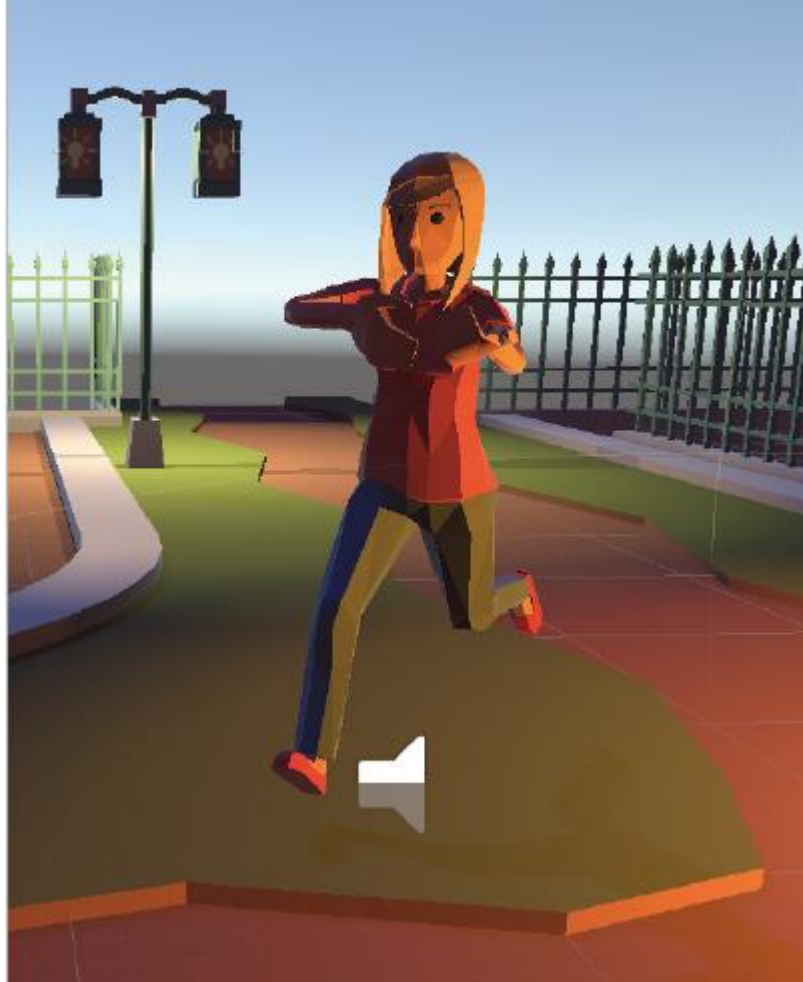
14.4 캐릭터 이동 구현(22)

◀ 앞쪽에 이어

```
// 입력값에 따라 캐릭터를 좌우로 회전
private void Rotate() {
    // 상대적으로 회전할 수치 계산
    float turn = playerInput.rotate * rotateSpeed * Time.deltaTime;
    // 리지드바디를 이용해 게임 오브젝트 회전 변경
    playerRigidbody.rotation =
        playerRigidbody.rotation * Quaternion.Euler(0, turn, 0f);
}
}
```

14.4 캐릭터 이동 구현(23)

- PlayerMovement 스크립트 동작 테스트

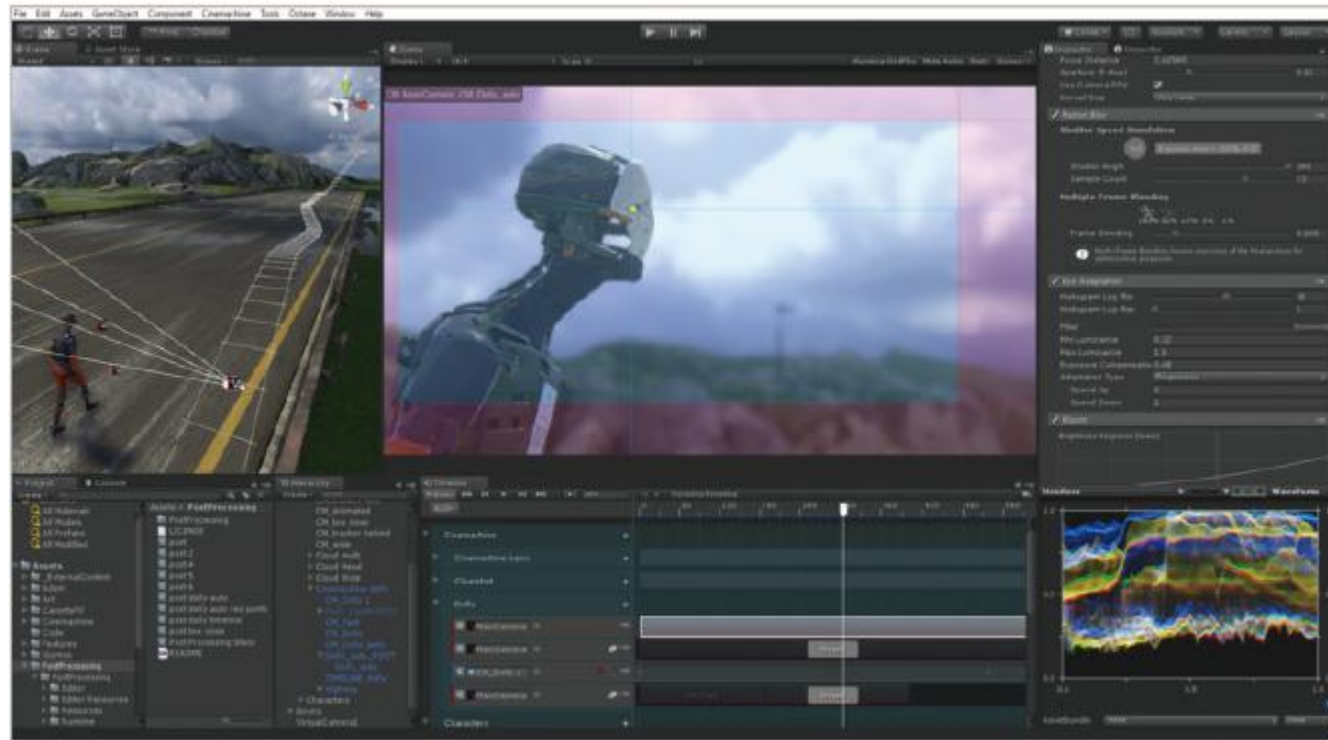


▶ 움직이는 플레이어 캐릭터

14.5 시네머신 추적 카메라 구성하기(1)

14.5.1 시네머신 소개

- 시네머신은 카메라의 움직임을 손쉽게 제어하는 유니티 공식 패키지
 - 카메라 연출에 필요한 코드와 조정 작업 대부분을 대체
 - 시네머신을 사용하면 레이싱, 어드벤처, TPS 등 장르마다 고유한 카메라 동작을 별다른 스크립트 작성 없이 구현
 - 게임 플레이뿐만 아니라 게임 컷신에서도 많은 부분을 자동화



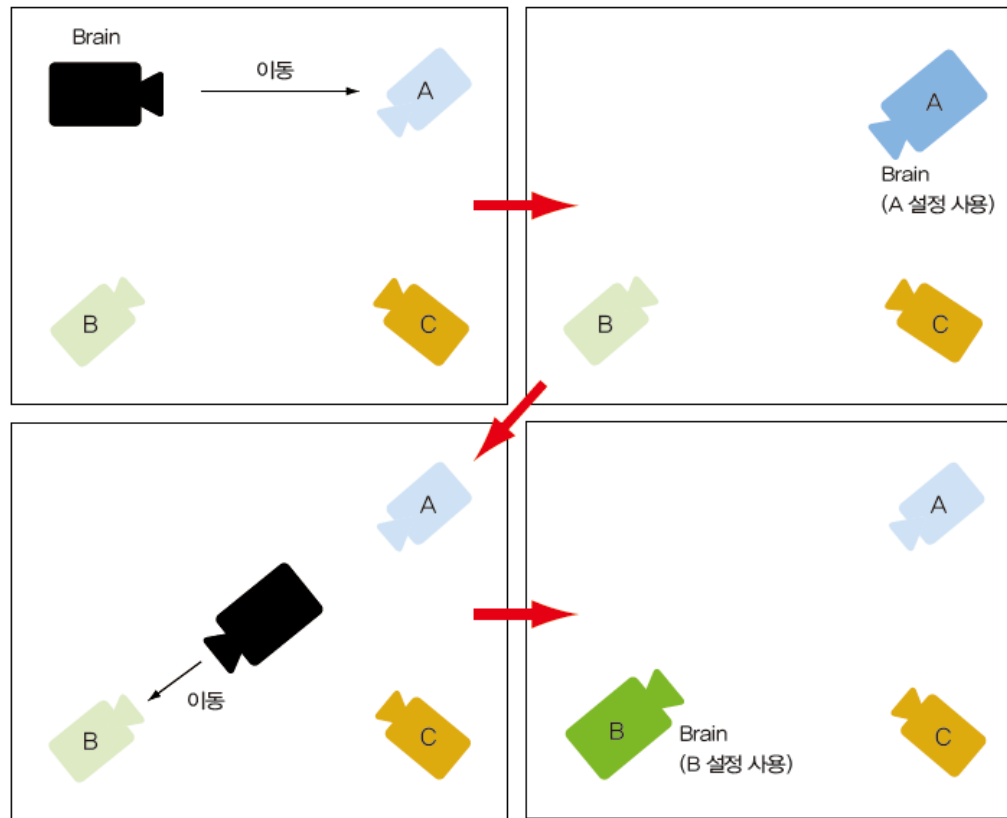
▶ 시네머신과 타임라인 에디터를 함께 사용한 모습

14.5 시네머신 추적 카메라 구성하기(2)

◦ 14.5.2 시네머신의 원리

- 시네머신이 제공하는 카메라

- 브레인 카메라(Brain Camera) - 게임 월드를 촬영하는 '진짜' 카메라이며 씬에 하나만 존재
- 가상 카메라(Virtual Camera) - 브레인 카메라의 '분신' 역할을 하며 씬에 여러 개 존재



▶ 시네머신 카메라의 동작 원리

14.5 시네머신 추적 카메라 구성하기(3)

14.5.3 추적 카메라 만들기

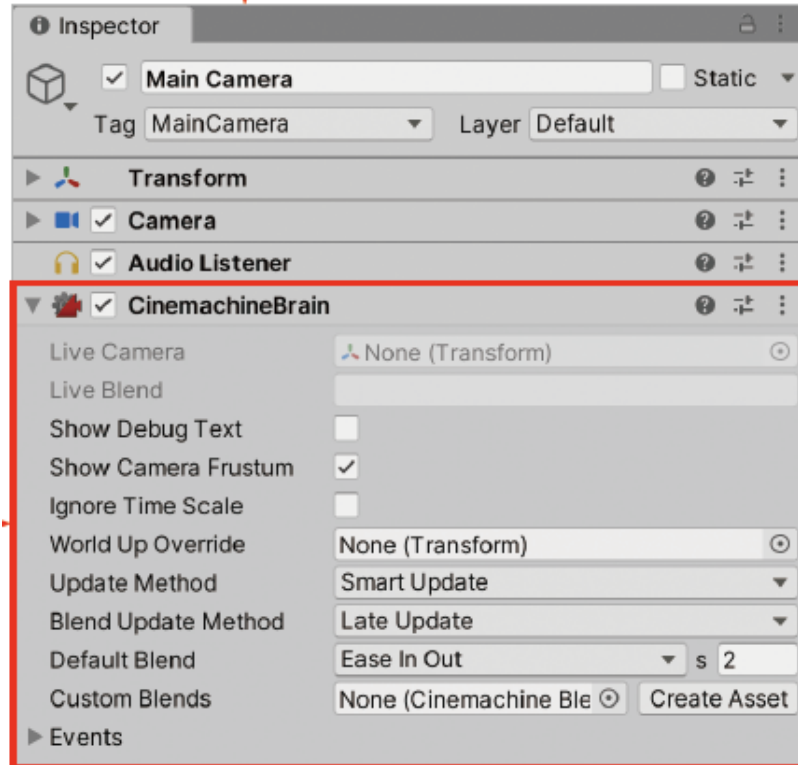
- 현재 씬에 있는 Main Camera 게임 오브젝트를 브레인 카메라로 만들고 브레인 카메라가 사용할 가상 카메라도 하나 만들기

[과정 01] 브레인 카메라와 가상 카메라 만들기

- ① 하이어라키 창에서 Main Camera 게임 오브젝트 선택
- ② Cinemachine Brain 컴포넌트 추가(Add Component > Cinemachine > Cinemachine Brain)
- ③ 새 가상 카메라 생성(하이어라키 창에서 + > Cinemachine > Virtual Camera 클릭)
- ④ 생성된 가상 카메라 게임 오브젝트의 이름을 Follow Cam으로 변경

14.5 시네머신 추적 카메라 구성하기(4)

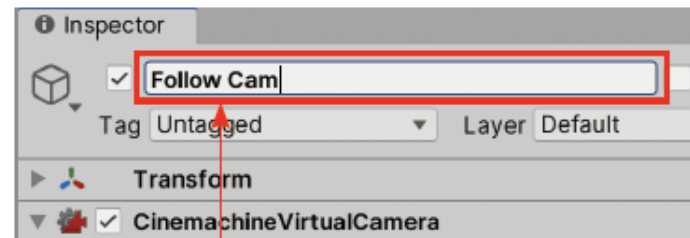
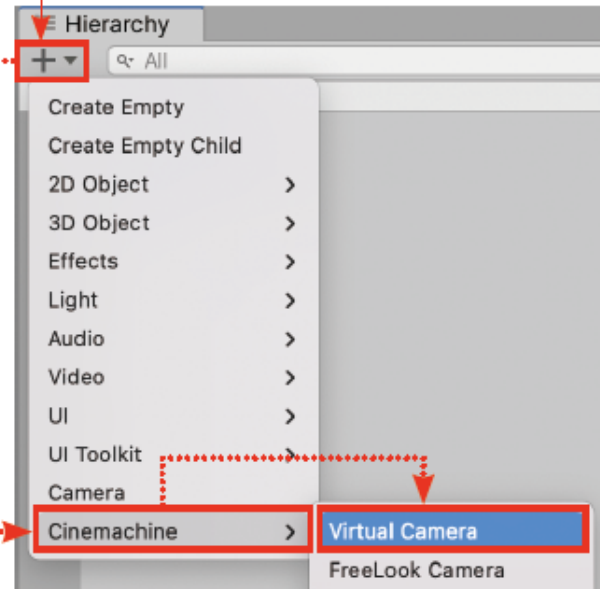
① Main Camera 게임 오브젝트 선택



② Cinemachine Brain 컴포넌트 추가(Add Component > Cinemachine > Cinemachine Brain)

▶ 브레인 카메라와 가상 카메라 만들기

③ 새 가상 카메라 생성(하이어라키 창에서 + > Cinemachine > Virtual Camera 클릭)



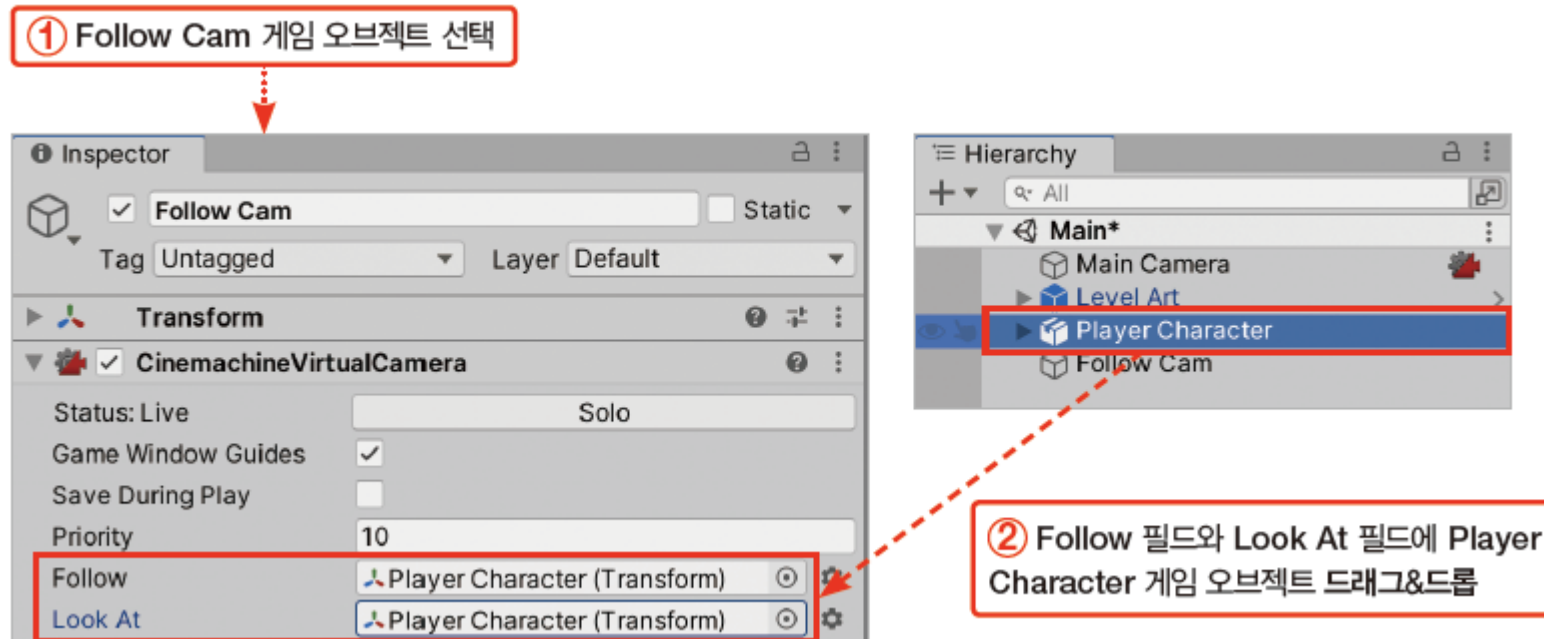
④ 생성된 가상 카메라 게임 오브젝트의 이름을 Follow Cam으로 변경

14.5 시네머신 추적 카메라 구성하기(5)

- 가상 카메라 Follow Cam이 플레이어 캐릭터를 추적하도록 설정

[과정 02] Follow Cam의 추적 대상 설정하기

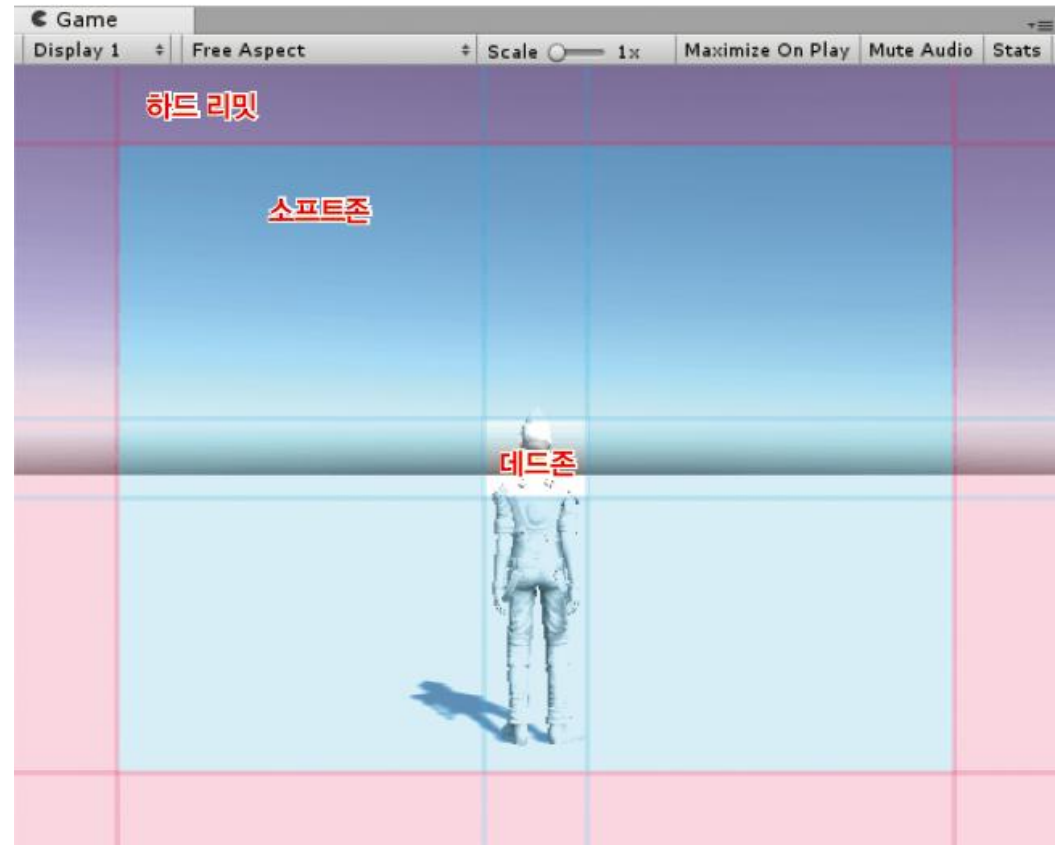
- ① 하이어라키 창에서 Follow Cam 게임 오브젝트 선택
- ② Cinemachine Virtual Camera 컴포넌트의 Follow 필드와 Look At 필드에 Player Character 게임 오브젝트 드래그&드롭



14.5 시네머신 추적 카메라 구성하기(6)

14.5.4 데드존, 소프트존, 하드 리밋

- 시네머신 가상 카메라 컴포넌트의 Look At 필드에 플레이어 캐릭터를 할당하는 순간 게임 창에 붉은색과 푸른색으로 그려진 영역과 안내선이 보임
- 데드 존, 소프트존, 하드 리밋은 카메라의 자연스러운 추적을 구현하는 데 사용
- 데드존에 있으면 카메라 회전 X
- 소프트존에 있으면 물체가 화면의 조준점 aim에 오도록 부드럽게 회전
- 물체가 빨리 움직여 화면의 소프트존을 벗어나 하드 리밋에 도달하려 하면 카메라가 격하게 회전
- 데드존/소프트존의 크기를 작게하면 물체가 조금이라도 화면 중앙을 벗어나려 할때 즉시 움직임
→ 화면의 움직임이 딱딱하게 느껴질 수 있음
- 데드존/소프트존의 크기를 크게하면 화면의 움직임이 느리게 느껴질 수 있음



14.5 시네머신 추적 카메라 구성하기(7)

14.5.5 Body와 Aim 설정

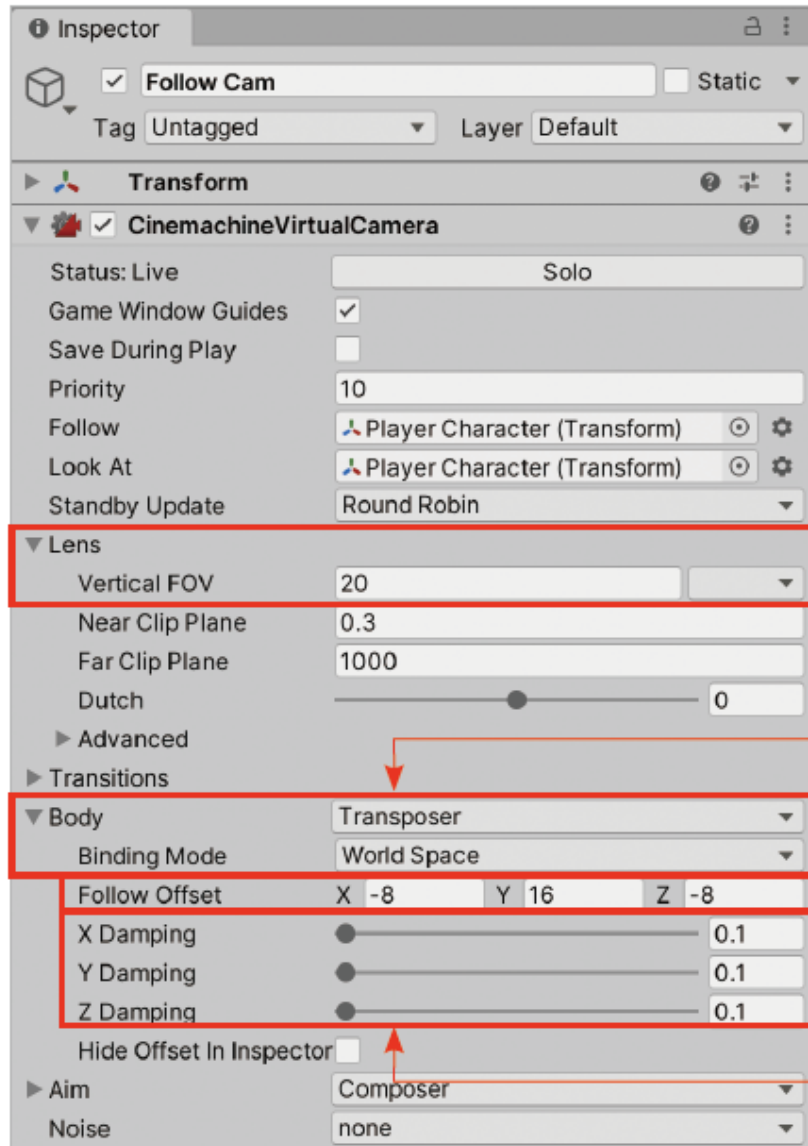
- 카메라의 데드존과 소프트존을 최대한 작게 만들어 캐릭터를 빠르게 추적
- 시네머신 가상 카메라 컴포넌트의 값을 조정

[과정 01] 가상 카메라 시야각과 Body 파라미터 설정하기

- ① Field Of View를 20으로 변경
- ② Body 탭 펼치기 > Binding Mode를 World Space로 변경
- ③ Follow Offset을 (-8, 16, -8)로 변경
- ④ X Damping, Y Damping, Z Damping을 0.1로 변경

- 시야각(Field of View, FOV)을 줄여 게임 화면을 좁힌
 - Body 파라미터는 카메라가 Follow에 할당된 추적 대상을 어떻게 따라다닐지 결정
- 카메라는 추적 대상인 Player Character 게임 오브젝트를 (-8, 16, -8)만큼 거리를 두고 추적
- 제동(Damping)값은 값의 급격한 변화를 '꺾어' 이전 값과 이후 값을 부드럽게 이어주는 비율
제동 damping을 크게하면 카메라 위치의 급격한 변화는 줄지만 위치가 신속하게 변경되지 않아 지연시간이 늘어남, 0.1로 줄여 신속하게 쫓아가게 함

14.5 시네머신 추적 카메라 구성하기(8)



① Field Of View를 20으로 변경

② Body 탭 펼치기 > Binding Mode를 World Space로 변경

③ Follow Offset을 (-8, 16, -8)로 변경

④ X Damping, Y Damping, Z Damping을 0.1로 변경

14.5 시네머신 추적 카메라 구성하기(9)

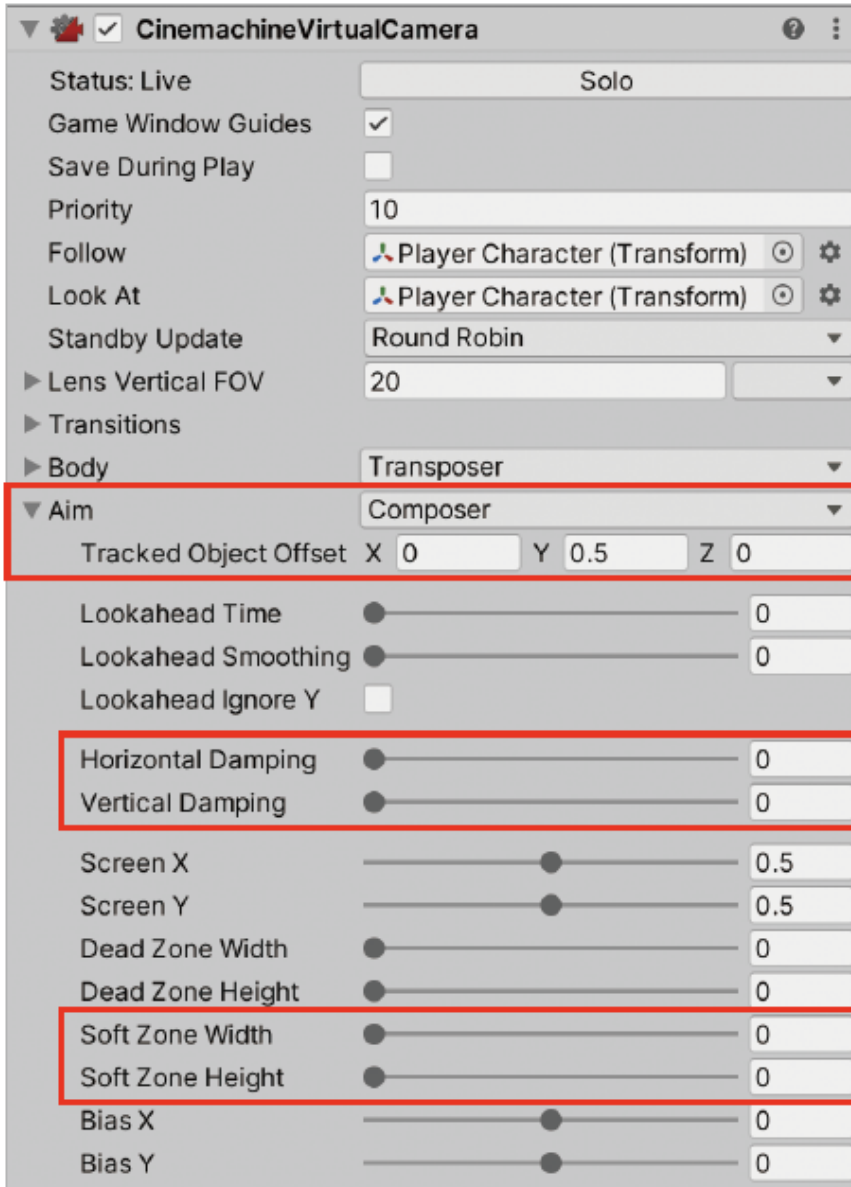
- 카메라가 조준하는 지점을 설정하는 Aim 파라미터 값을 수정

[과정 02] 가상 카메라의 Aim 파라미터 설정하기

- ① Tracked Object Offset을 (0, 0.5, 0)으로 변경
- ② Horizontal Damping과 Vertical Damping을 0으로 변경
- ③ Soft Zone Width와 Soft Zone Height를 0으로 변경

- 기본적으로 가상 카메라는 Look At에 할당된 게임 오브젝트를 항상 조준하도록 회전
 - Tracked Object Offset 필드는 원래 추적 대상에서 얼마나 더 떨어진 곳을 조준할지 결정
 - Tracked Object Offset의 Y 값을 조금 늘려 카메라가 플레이어 캐릭터의 실제 위치보다 조금 높은 곳을 조준하게 변경
- 회전 속도에 대한 제동값을 가로와 세로 방향 모두 0으로 변경하고, 소프트존의 가로와 세로 크기를 모두 0으로 변경하여 소프트존을 없애버림
 - 따라서 카메라가 지연시간 없이 Player Character 게임 오브젝트를 즉시 조준

14.5 시네머신 추적 카메라 구성하기(10)



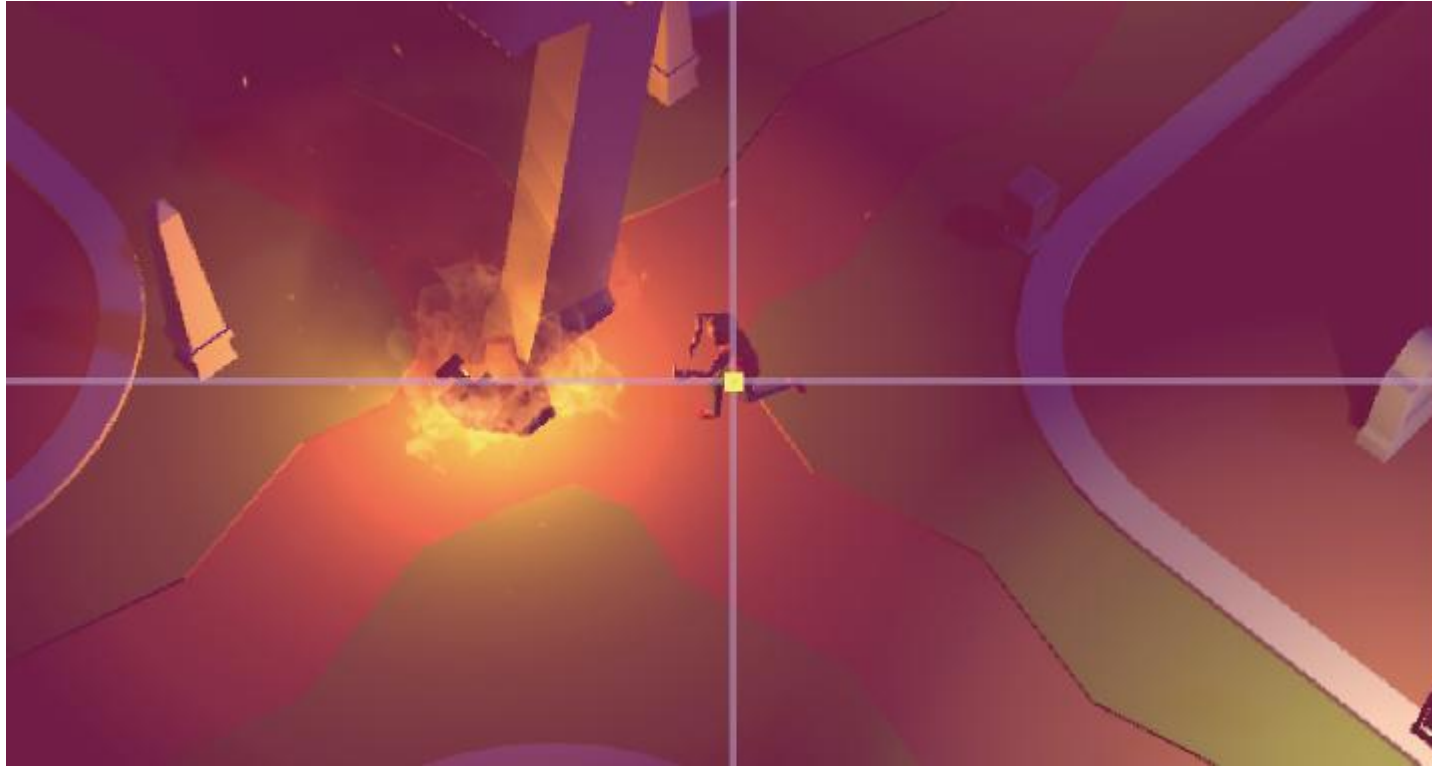
① Aim 탭 펼치기 > Tracked Object Offset을 (0, 0.5, 0)으로 변경

② Horizontal Damping과 Vertical Damping을 0으로 변경

③ Soft Zone Width와 Soft Zone Height를 0으로 변경

14.5 시네머신 추적 카메라 구성하기(11)

- 플레이 버튼을 눌러 씬을 테스트



▶ 추적 카메라 완성

14.6 마치며

이 장에서 배운 내용 요약

- 유니티 패키지 매니저를 사용해 유니티 공식 패키지를 다운로드
- 유니티 패키지 매니저는 중앙 저장소 패키지들을 다운로드
- 라이트맵을 사용하면 적은 실시간 연산 비용으로 라이트 효과를 구현
- 글로벌 일루미네이션으로 간접광을 구현
- 환경광을 설정하면 게임의 전체 색 분위기를 조절
- 블렌드 트리를 사용해 여러 애니메이션 클립을 섞어서 사용
- 애니메이터 레이어를 여러 개 만들어 동시에 여러 상태 머신을 동작시킴
- 애니메이터 레이어는 위에서 아래 순서로 적용
- 아바타 마스크를 사용해 애니메이션을 3D 모델의 특정 부위에만 적용
- 입력과 액터를 나누면 사용자 입력을 나중에 개선하기 편리
- 프로퍼티를 사용하면 값을 가져오거나 새 값을 할당할 때 중간 처리를 삽입 가능
- 자동 구현 프로퍼티를 사용해 프로퍼티를 간결하게 생성
- 리지드바디의 MovePosition() 메서드는 입력으로 전역 위치를 받음
- 시네머신으로 다양한 종류의 카메라 동작을 손쉽게 생성
- 시네머신의 브레인 카메라는 실제 화면을 촬영하는 카메라
- 시네머신의 가상 카메라는 브레인 카메라의 분신 역할
- 브레인 카메라는 여러 개의 가상 카메라 중 하나를 현재 카메라로 사용
- 시네머신 가상 카메라의 Follow에는 카메라가 움직이면서 추적할 대상을 할당
- 시네머신 가상 카메라의 Look At에는 카메라가 회전하면서 조준할 대상을 할당
- 시네머신 카메라의 데드존, 소프트존, 하드 리미트로 카메라 추적의 강도를 조절